Introduction to Deep Learning

sjpark@rcv.sejong.ac.kr

Contents

1. What is Neural Network
    1) Machine Learning vs Deep Learning
    2) Neural Network
    3) Forward / Backward Pass
    4) Optimization
    5) Gradient Vanishing
    6) Universal Approximation Theorem

2. What is Convolutional Neural Network (CNN)
    1) Convolution
    2) Image Classification Pipeline
    3) Characteristics of CNN
        - Locality
        - Invariance
        - Equivariance

Contents

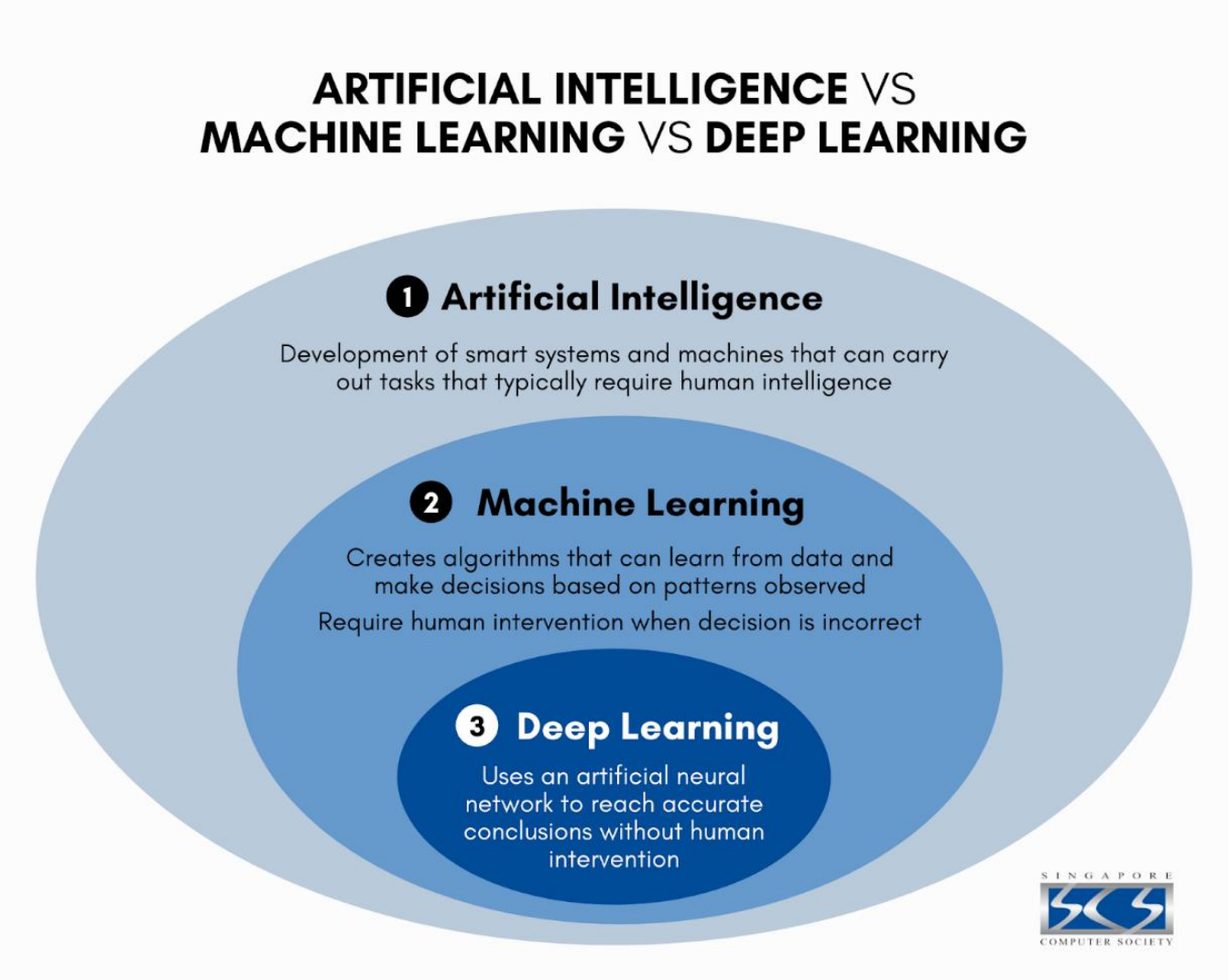
3. Training Techniques
1) Overfitting / Underfitting
2) Validation
3) Regularization
4) Data Augmentation
5) Weight Initialization
6) Batch Normalization
7) Stochastic Gradient Descent

1. What is Neural Network

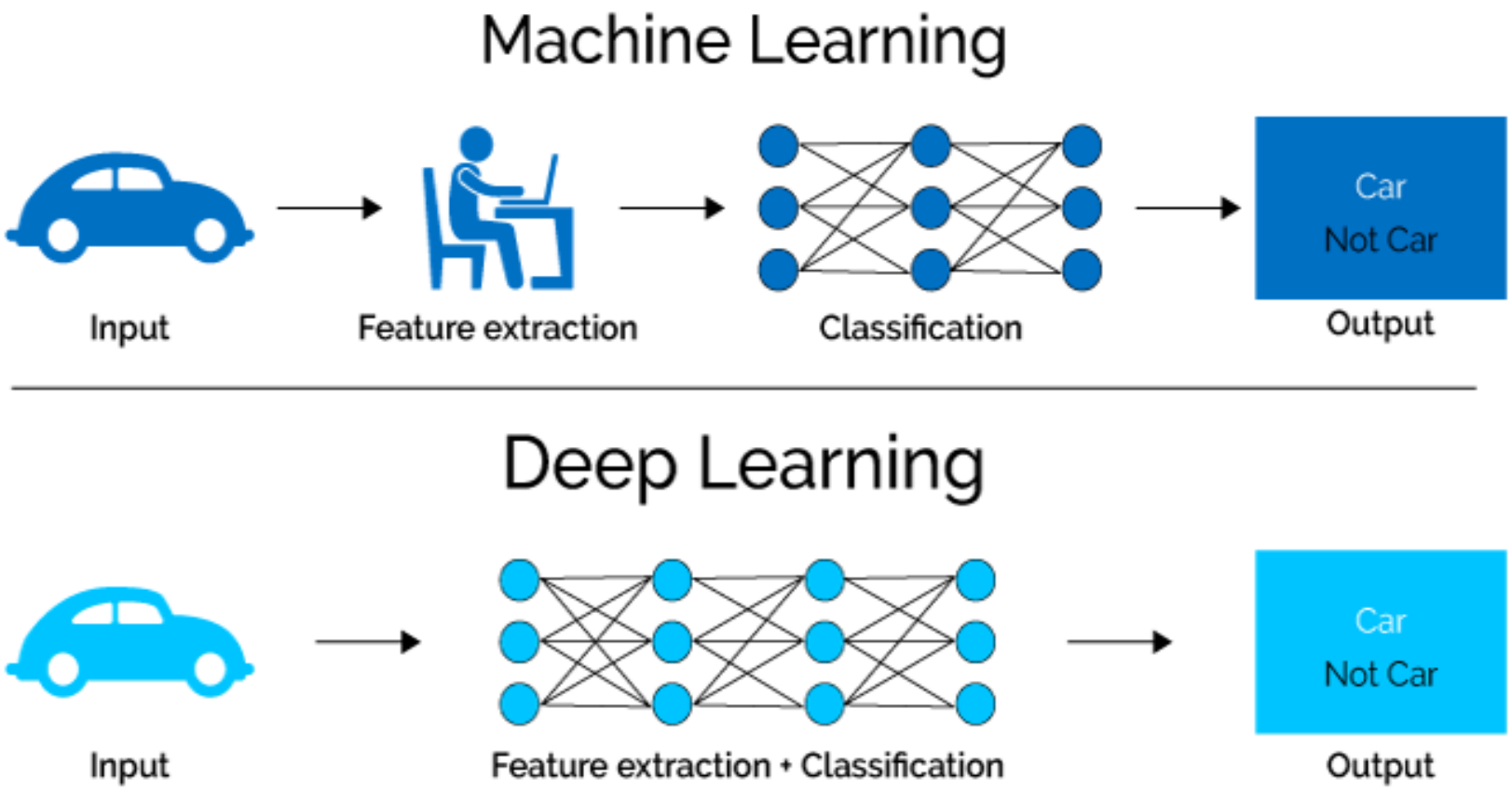
Contents

1. What is Neural Network
    1) Machine Learning vs Deep Learning
    2) Neural Network
    3) Forward / Backward Pass
    4) Optimization
    5) Gradient Vanishing
    6) Universal Approximation Theorem

1-1. Machine Learning vs Deep Learning

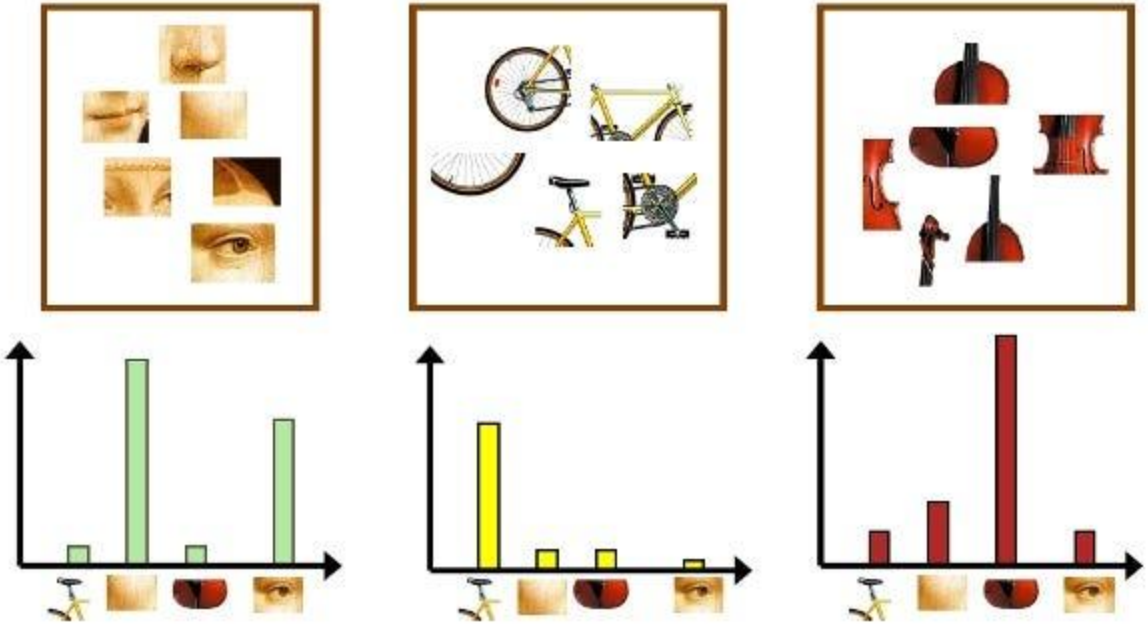


1-1. Machine Learning vs Deep Learning



1-1. Machine Learning vs Deep Learning

Example of Machine Learning: BoVW



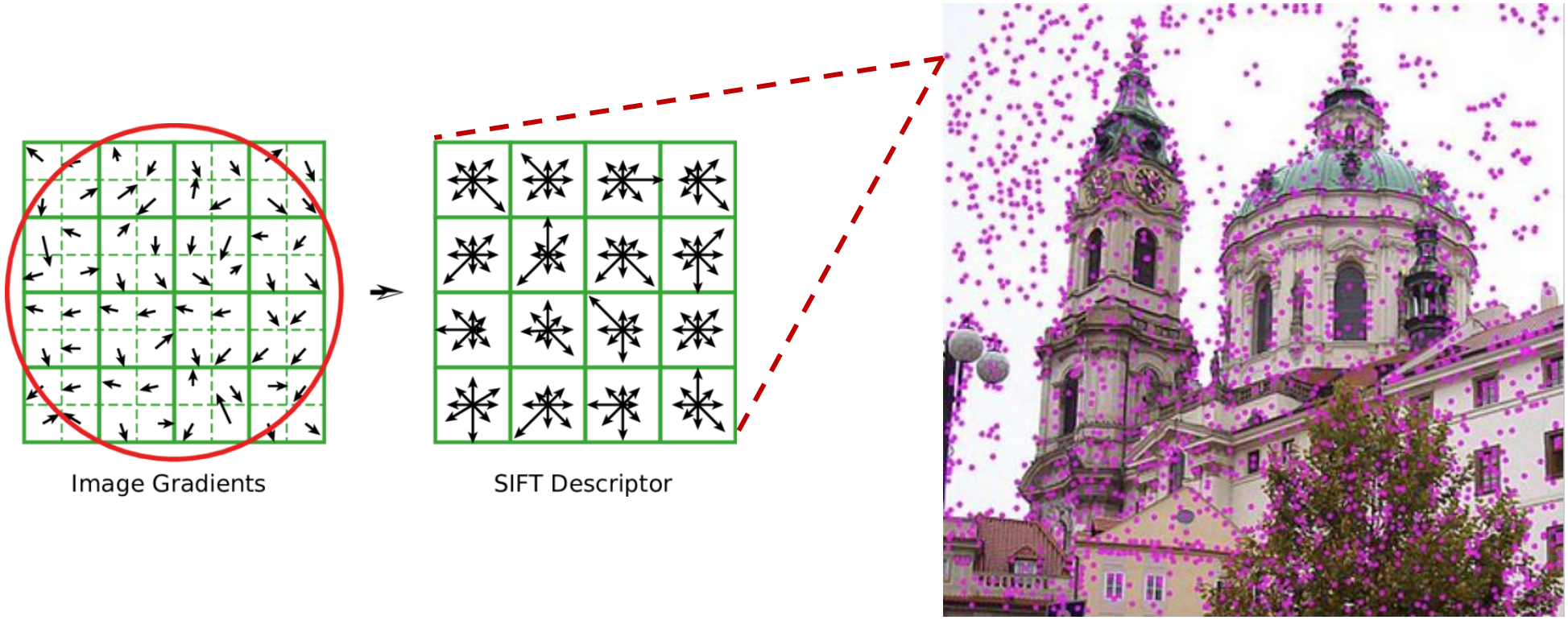
<Bag of Visual Word (BoVW)>

A definition of the BoVW model can be the
"**histogram representation based on independent features**"

1-1. Machine Learning vs Deep Learning

Example of Machine Learning: BoVW

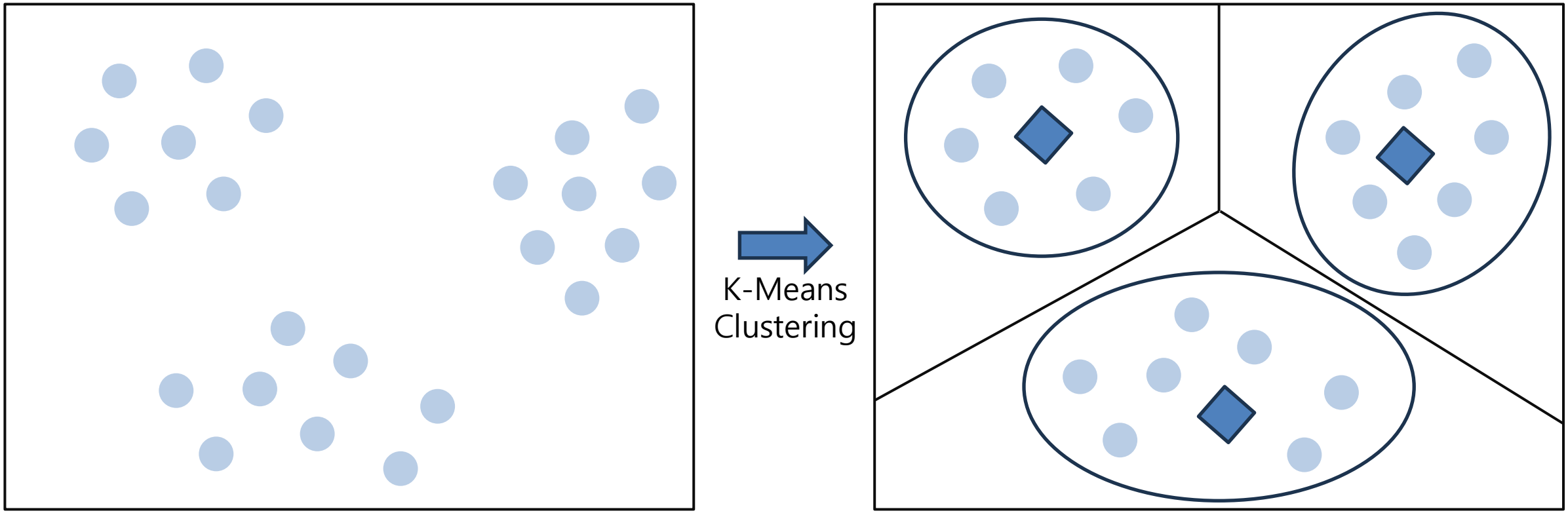
SIFT : Scale-Invariant Feature Transform



Step 1) Aggregate the **<u>SIFT descriptors</u>** of all the images

1-1. Machine Learning vs Deep Learning

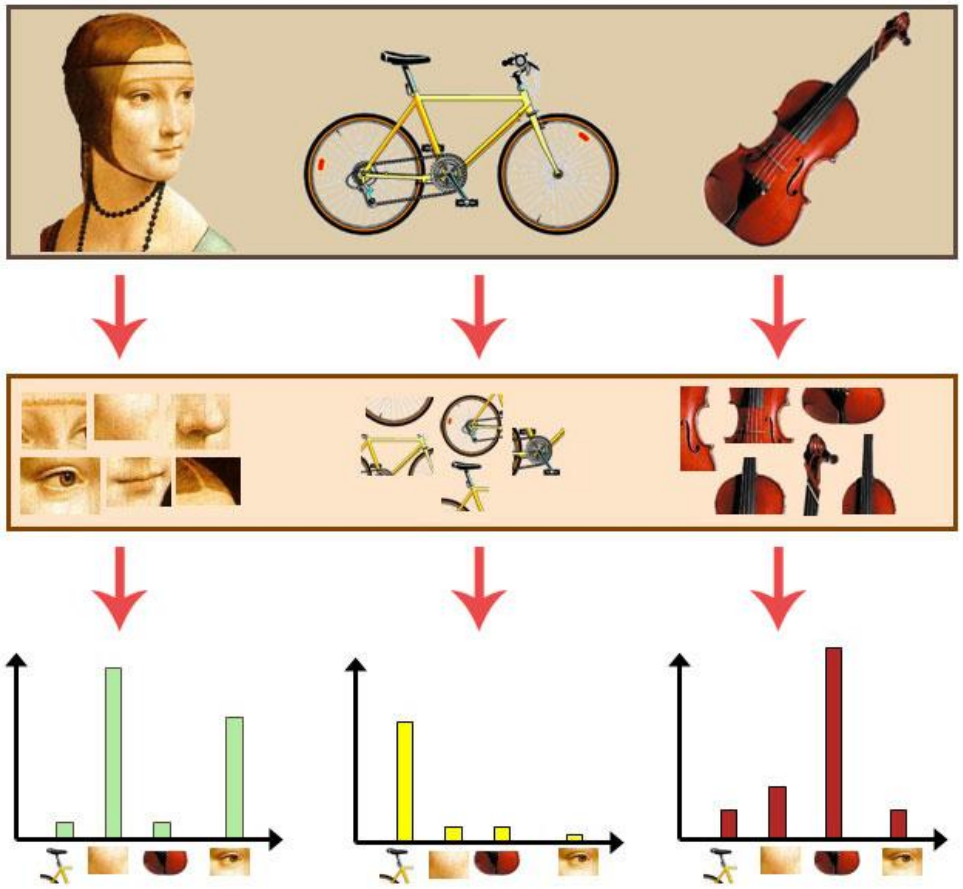
Example of Machine Learning: BoVW



Step 2) Generate Codebook via Clustering

1-1. Machine Learning vs Deep Learning

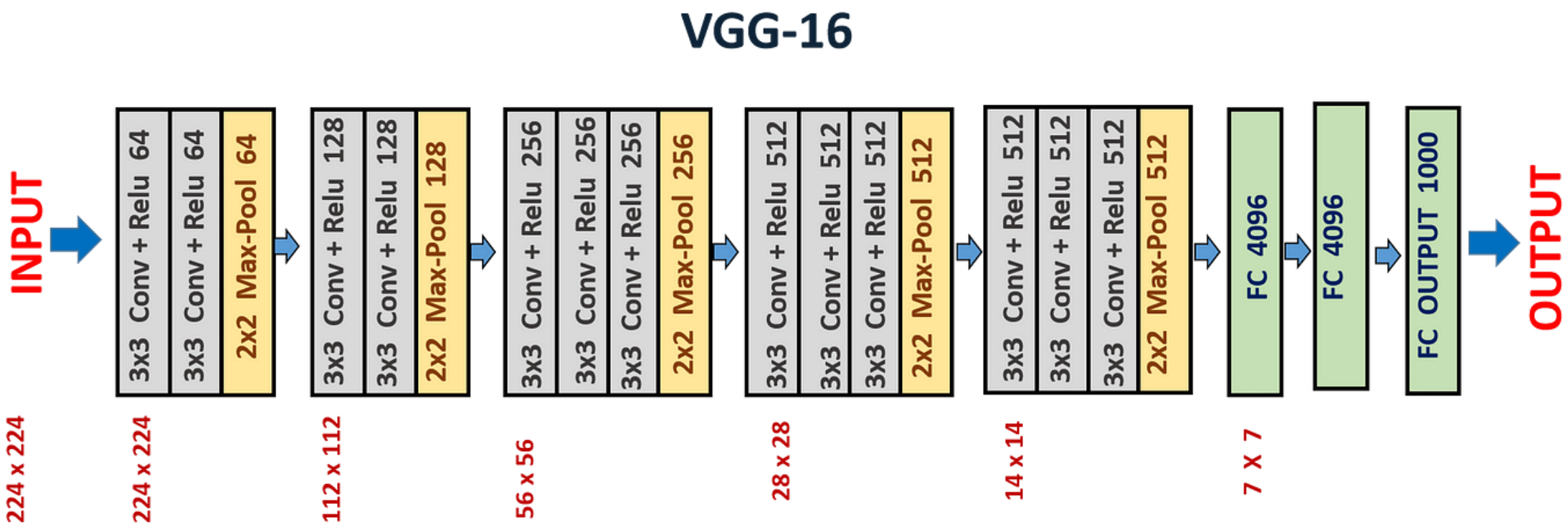
Example of Machine Learning: BoVW



Step 3) Represent image as histogram via codeword

1-1. Machine Learning vs Deep Learning

Example of Deep Learning: VGG16



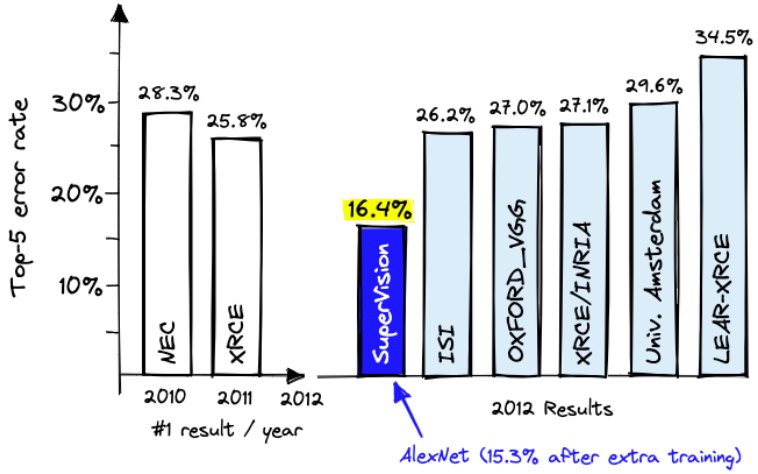
1-1. Machine Learning vs Deep Learning

ImageNet Large Scale Visual Recognition Challenges (ILSVRC)

ImageNet: 14 million images classified by WordNet hierarchy of nouns

1-1. Machine Learning vs Deep Learning

Example of Deep Learning: AlexNet



ILSVRC 2012

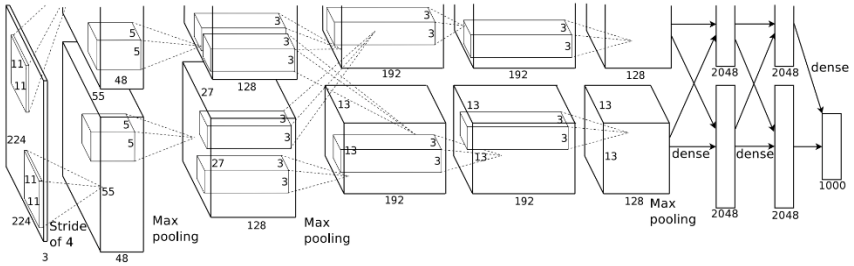


Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

AlexNet[1]

Task 1

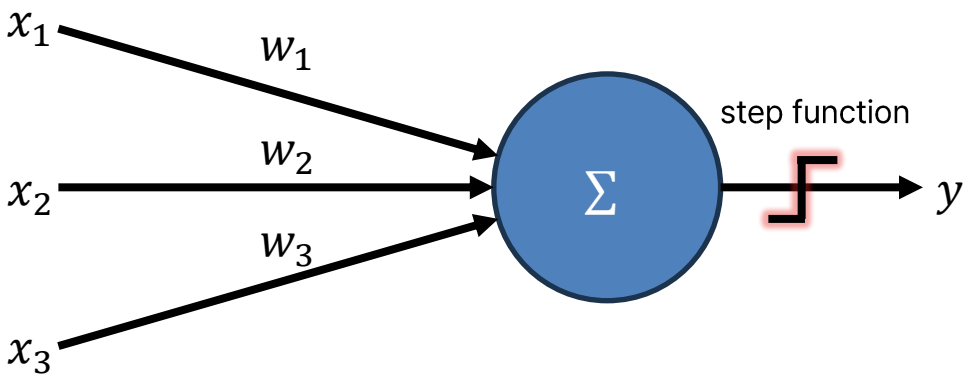
Team name	Filename	Error (5 guesses)	Description
SuperVision	test-preds-141-146.2009-131-137-145-146.2011-145f.	0.15315	Using extra training data from ImageNet Fall 2011 release
SuperVision	test-preds-131-137-145-135-145f.txt	0.16422	Using only supplied training data
ISI	pred_FVs_wLACs_weighted.txt	0.26172	Weighted sum of scores from each classifier with SIFT+FV, LBP+FV, GIST+FV, and CSIFT+FV, respectively.
ISI	pred_FVs_weighted.txt	0.26602	Weighted sum of scores from classifiers using each FV.

[1] ImageNet Classification with Deep Convolutional Neural Networks, Alex Krizhevsky, Ilya Sutskever, Geoffrey E. Hinton (2012, NIPS)

1-2. Neural Network

Perceptron (Linear Model)

- Type of **<u>linear classifier</u>**, i.e. a classification algorithm that makes its predictions based on a <u>linear predictor function combining a set of weights with the feature vector</u>

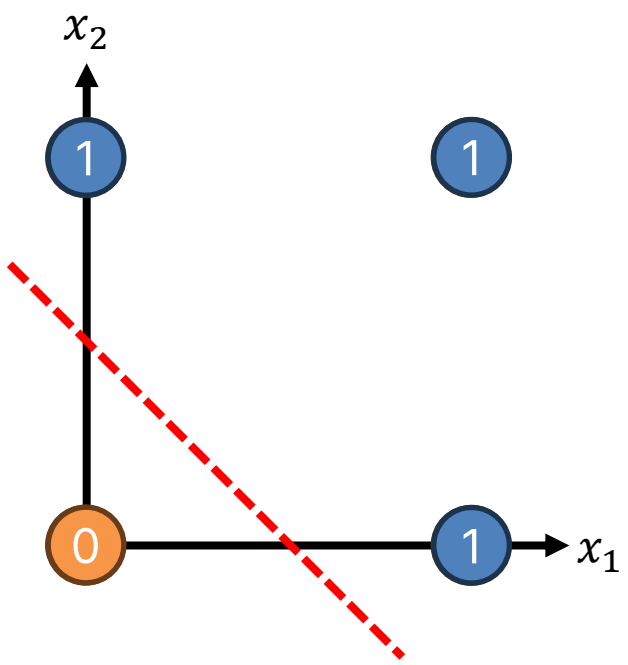


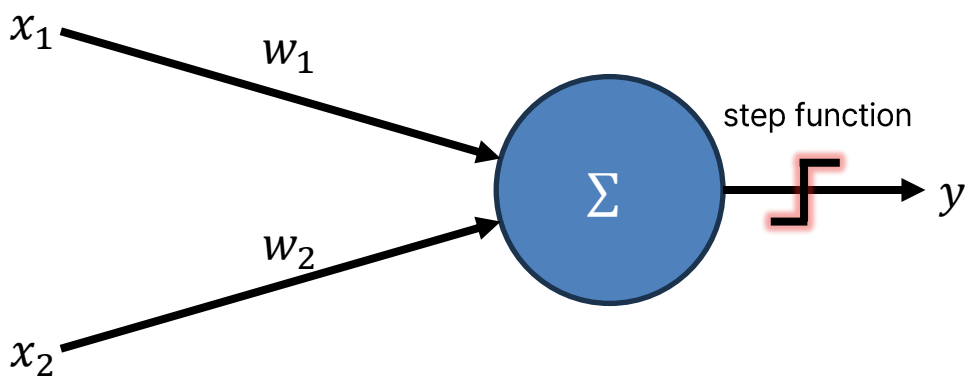
$$y = s\left(\sum_{i=0}^{N} w_i x_i\right)$$

$$s(v) = \begin{cases} 1 & \text{if } v > \theta \\ 0 & \text{otherwise} \end{cases}$$

1-2. Neural Network

Perceptron (Linear Model)





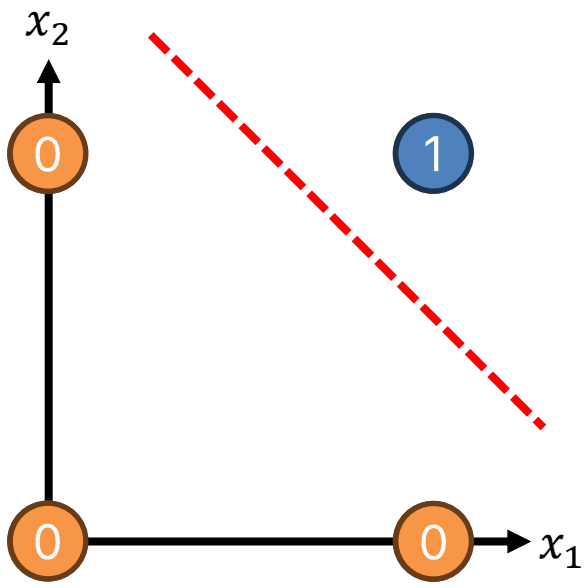
$$y = s\left(\sum_{i=0}^{N} w_i x_i\right)$$

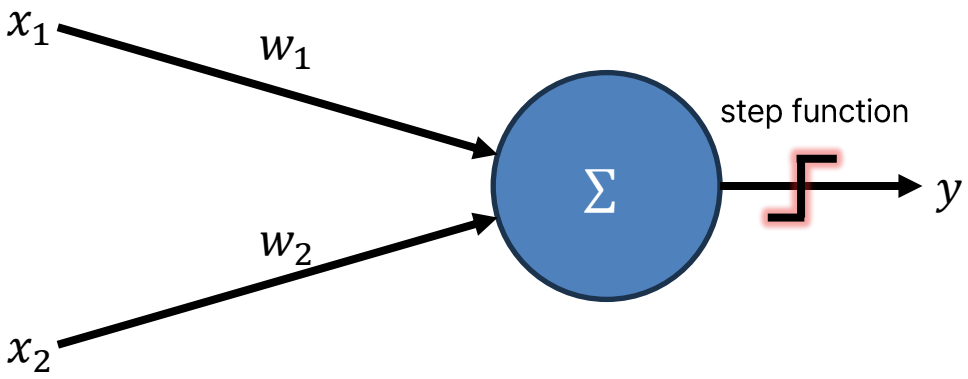
$$s(v) = \begin{cases} 1 & \text{if } v > 0.6 \\ 0 & \text{otherwise} \end{cases}$$

Let $w_1 = 0.7$ and $w_2 = 0.7$ … Perceptron works as OR gate!

1-2. Neural Network

Perceptron (Linear Model)





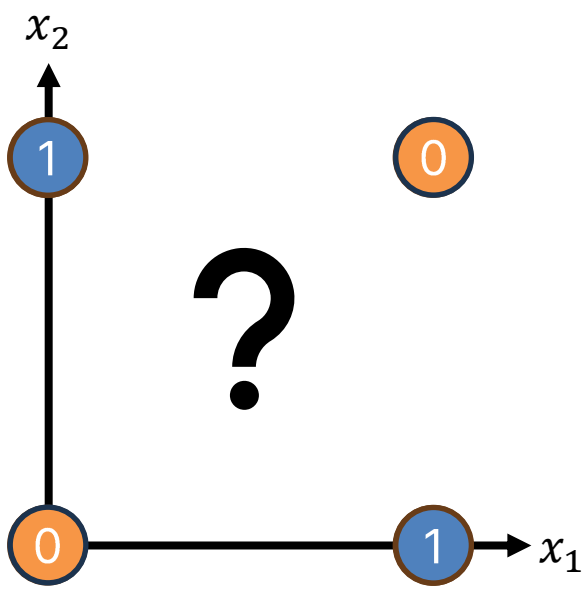
$$y = s\left(\sum_{i=0}^{N} w_i x_i\right)$$

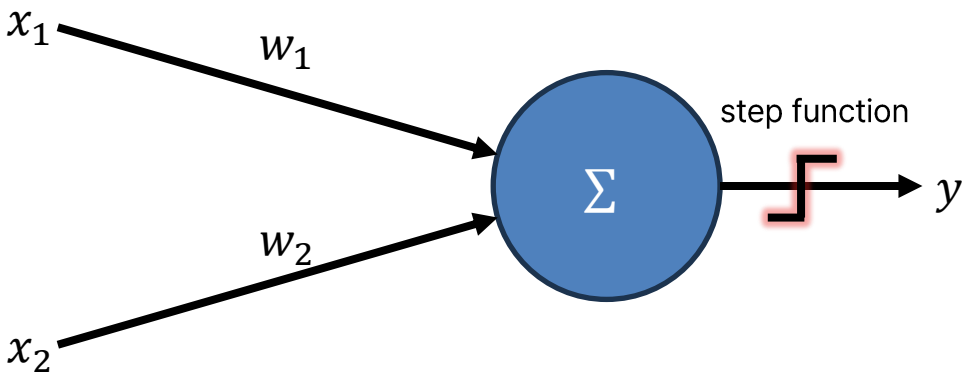
$$s(v) = \begin{cases} 1 & \text{if } v > 0.6 \\ 0 & \text{otherwise} \end{cases}$$

When $w_1 = 0.5$ and $w_2 = 0.5$ … It works as AND gate!

1-2. Neural Network

Perceptron (Linear Model)





$$y = s\left(\sum_{i=0}^{N} w_i x_i\right)$$

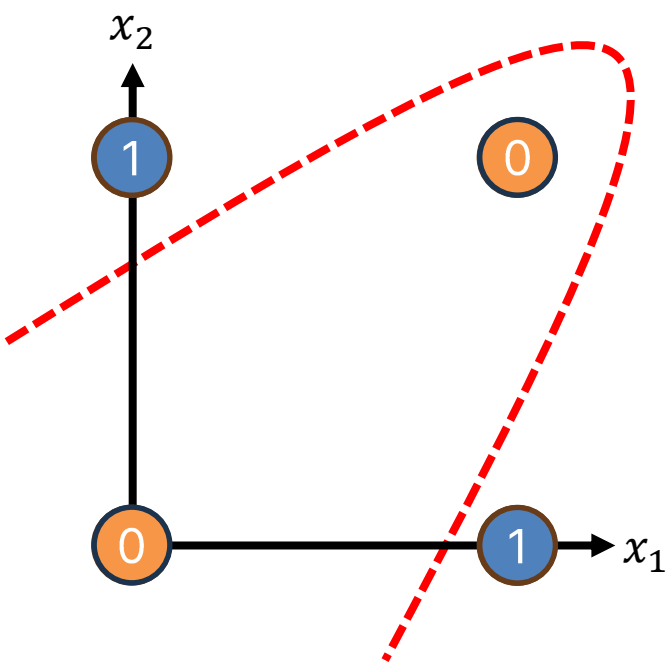
$$s(v) = \begin{cases} 1 & \text{if } v > 0.6 \\ 0 & \text{otherwise} \end{cases}$$

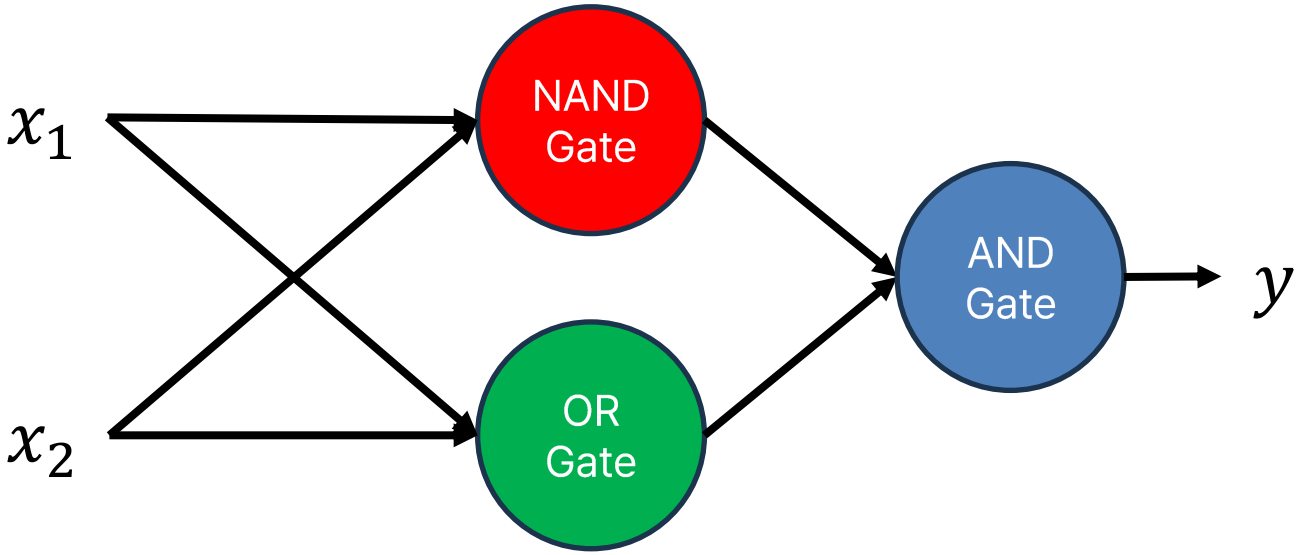
However, **<u>XOR</u>** could not be solved by a perceptron

1-2. Neural Network

Multi-Layer Perceptron (MLP)

- If the problem is not **<u>linearly separable</u>**, a single perceptron has its limitations

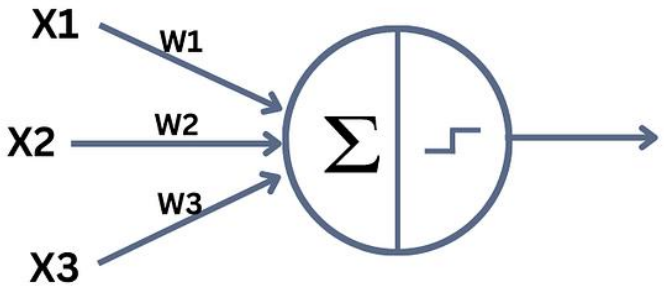




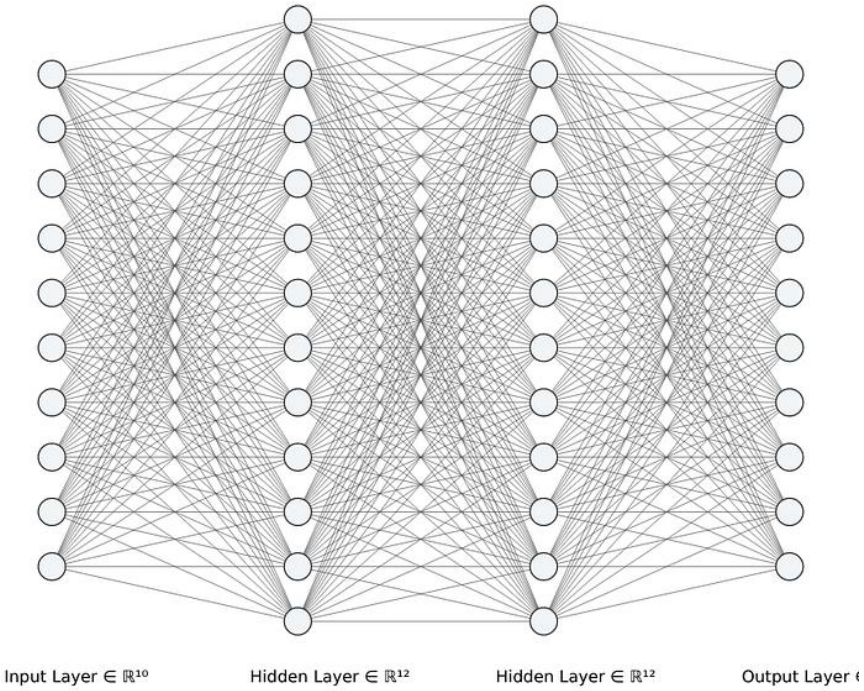
1-2. Neural Network

Multi-Layer Perceptron (MLP)

- With the **<u>continuous combination of linear units and non-linear functions,</u>** neural networks have begun to solve complex problems



Single-layer perceptron



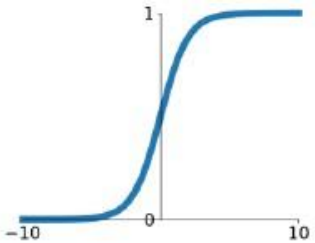
Multi-layer perceptron

1-2. Neural Network

Activation Function (Non-Linearity)

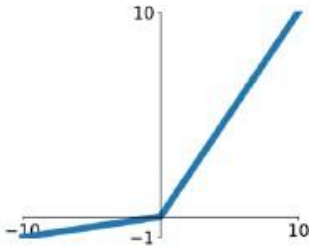
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



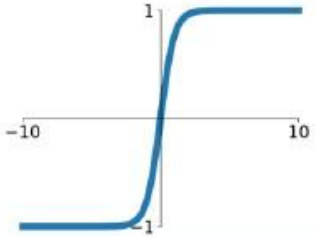
Leaky ReLU

$$\max(0.1x, x)$$



tanh

$$\tanh(x)$$

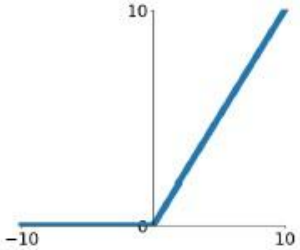


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

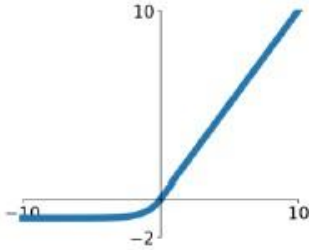
ReLU

$$\max(0, x)$$

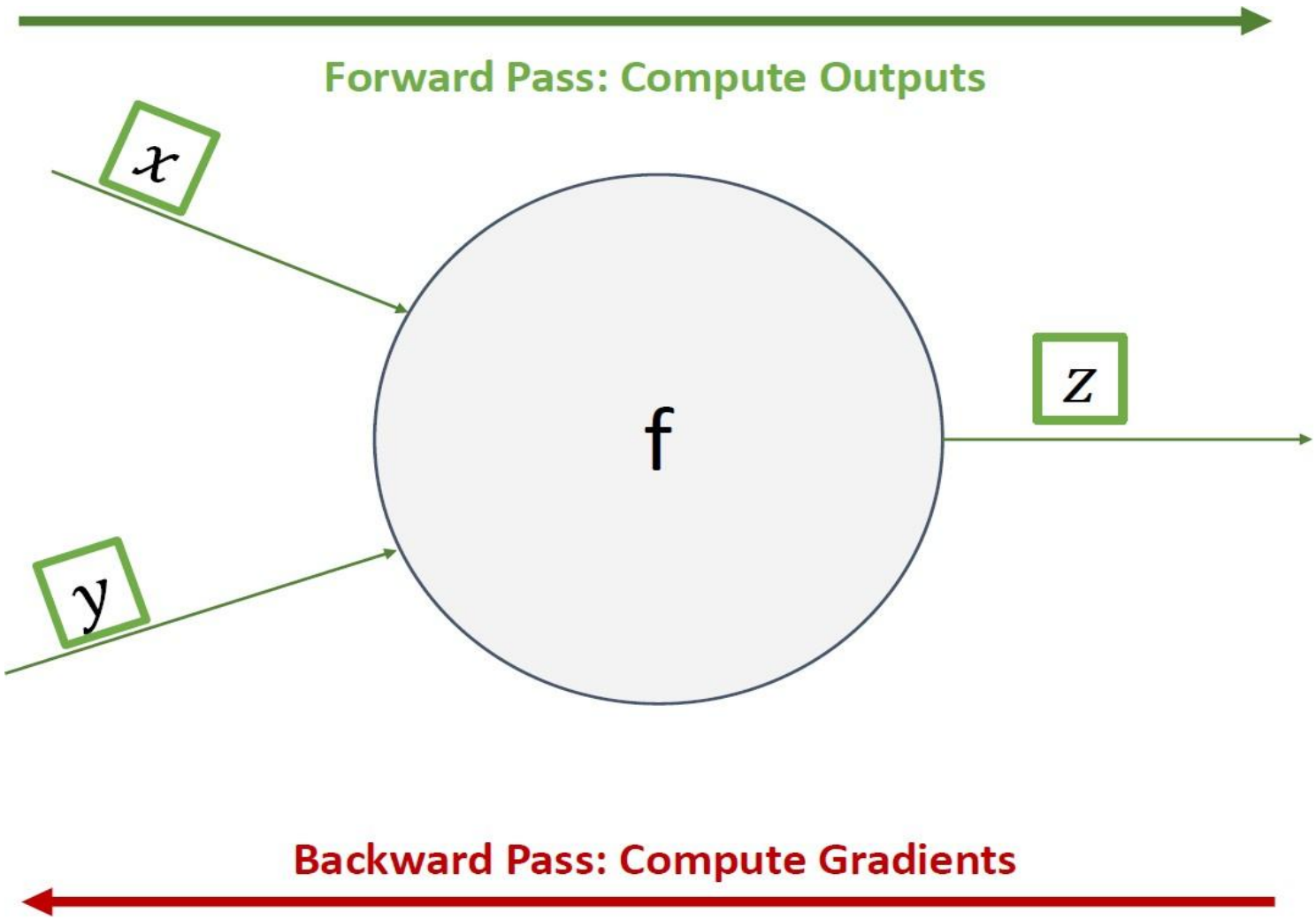


ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



1-3. Forward / Backward Pass



1-3. Forward / Backward Pass

Training Algorithm:

1. Initialize the weight parameter w with random values
2. Process the data and make prediction
3. Calculate the error between the network's predictions and the actual labels
4. Update the parameter w in the direction that reduces the error
5. Repeat 2~4 steps until the network's predictions become accurate

1-3. Forward / Backward Pass

Loss Function

A **loss function** tells how good our current classifier is

Low loss = good classifier
High loss = bad classifier

(Also called: **objective function**; **cost function**)

Given a dataset of examples

$$\{(x_i, y_i)\}_{i=1}^{N}$$

Where x_i is an vector (image) and
y_i is an integer-valued label

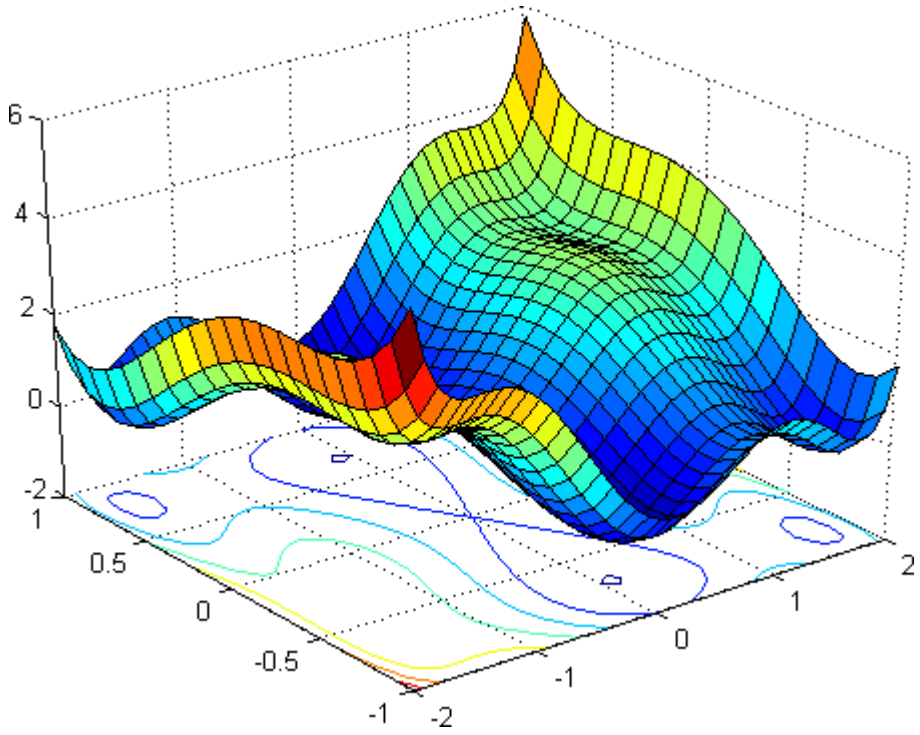
Loss for a single example is

$$L_i = L(f(x_i, W), y_i)$$

Loss for the dataset is average of per-example losses:

분류기의 Loss ➔

$$\phi(W) = \frac{1}{N}\sum_{i=1}^{N} L(f(x_i, W), y_i)$$



1-3. Forward / Backward Pass

Loss Function

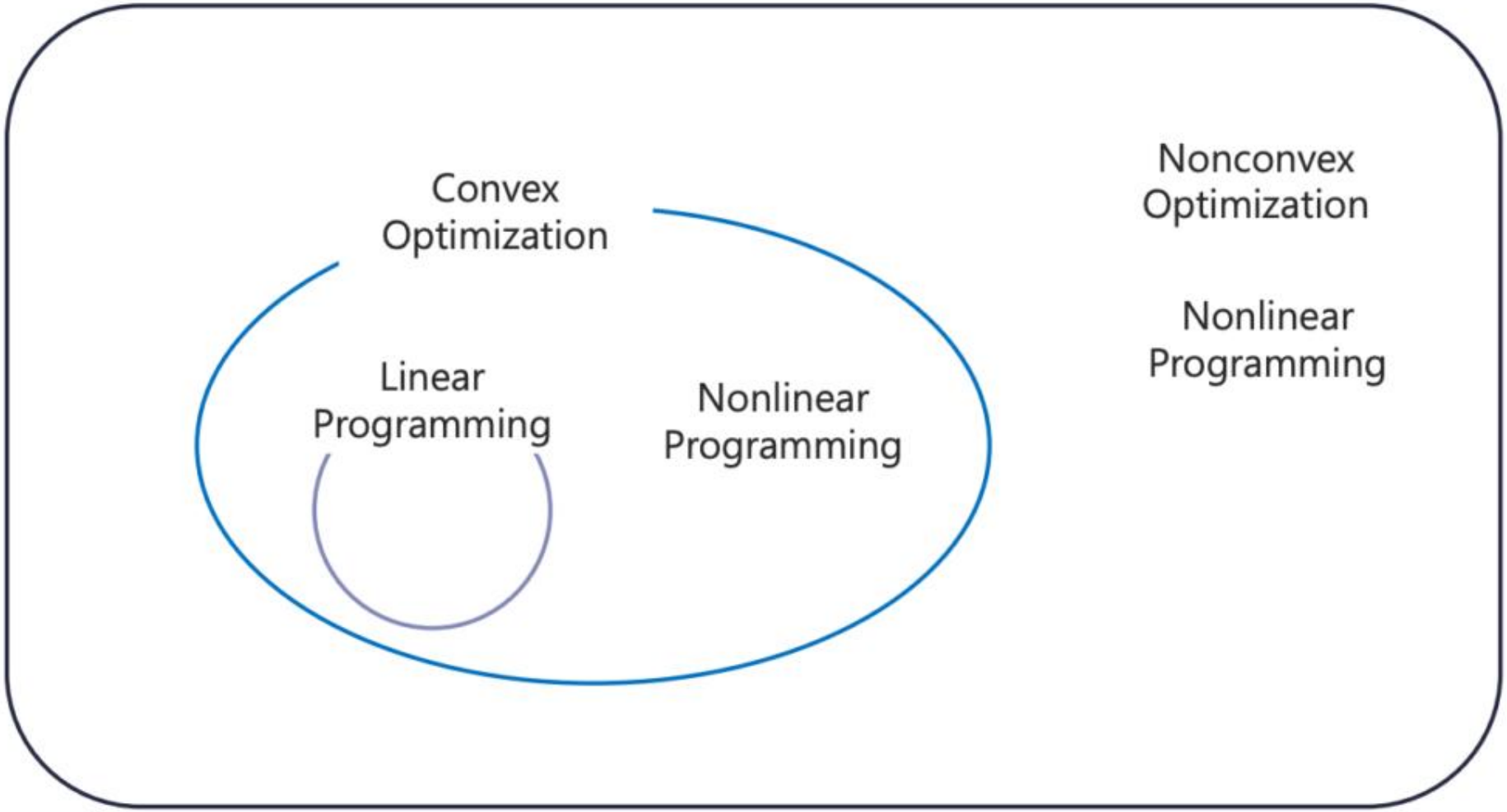
Regression Task

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}\sum_{d=1}^{D}(y_i^{(d)} - \hat{y}_i^{(d)})^2$$

Classification Task

$$\mathrm{CE} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{d=1}^{D} y_i^{(d)} \log \hat{y}_i^{(d)}$$

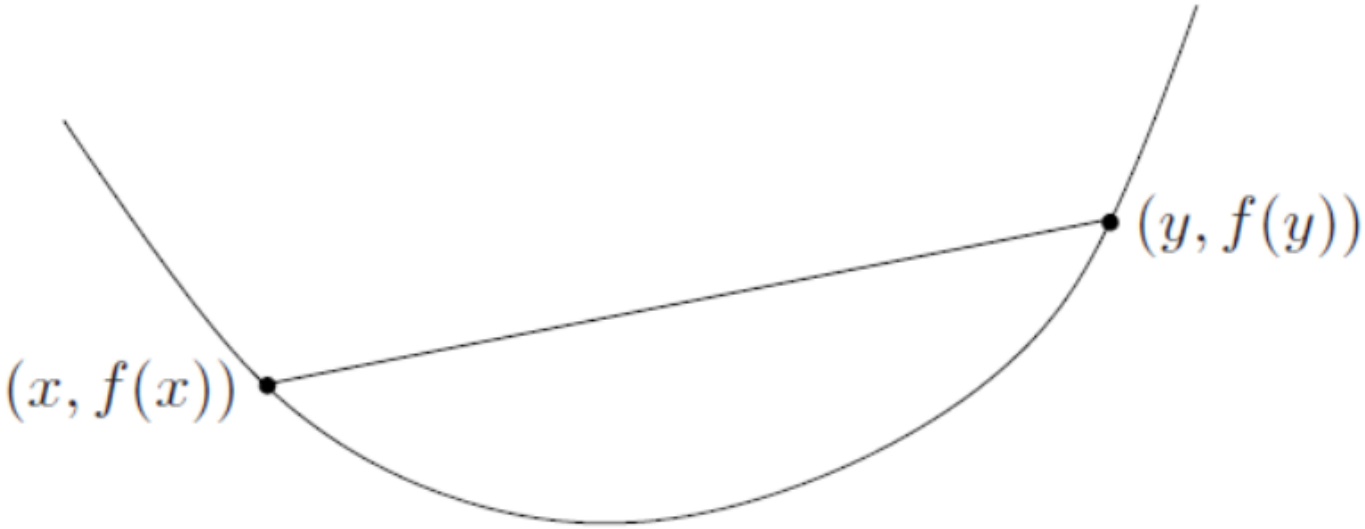
1-4. Optimization



1-4. Optimization

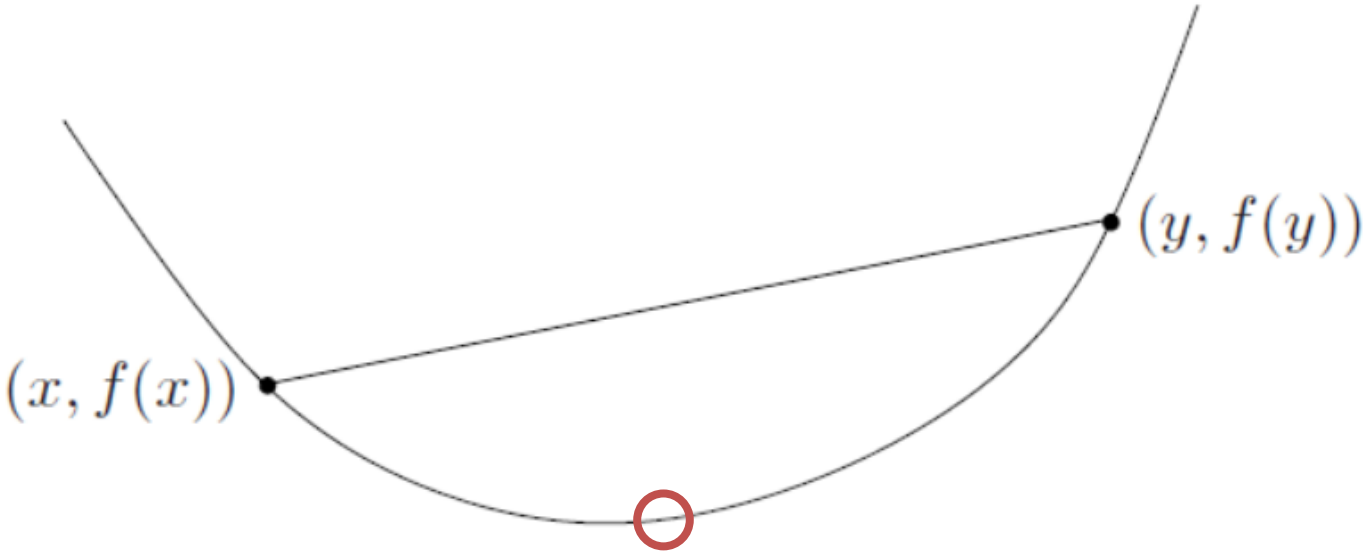
Convex function

$$f(\theta x + (1-\theta)y) \leq \theta f(x) + (1-\theta) f(y), 0 \leq \theta \leq 1$$



1-4. Optimization

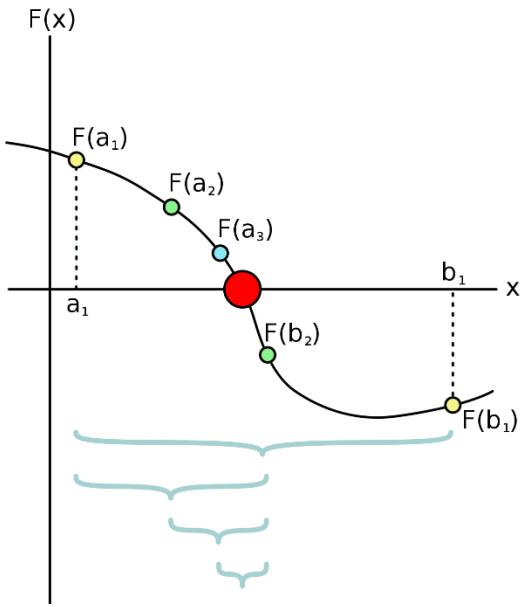
Convex function



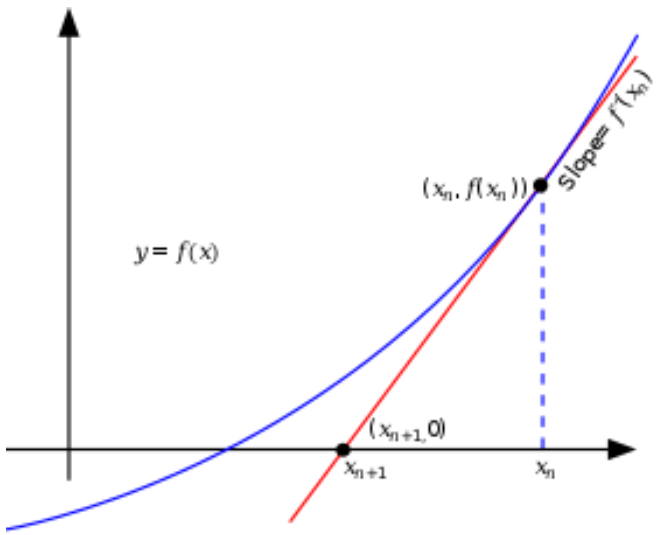
we just need to solve $\nabla f(x) = 0$

1-4. Optimization

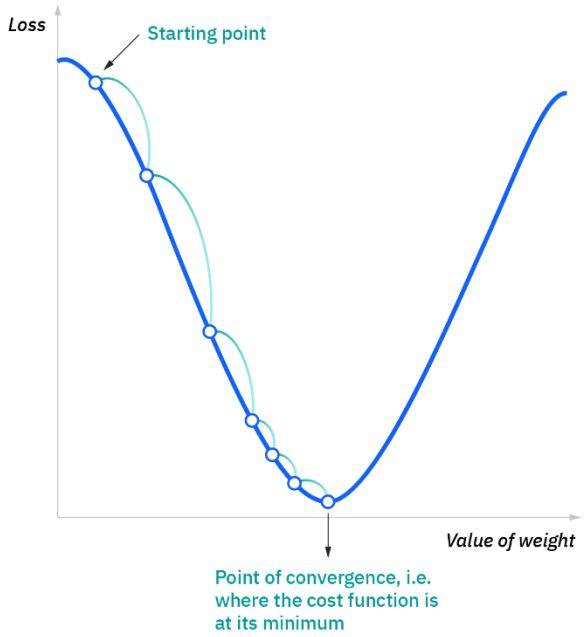
How to find minima?



Bisection method



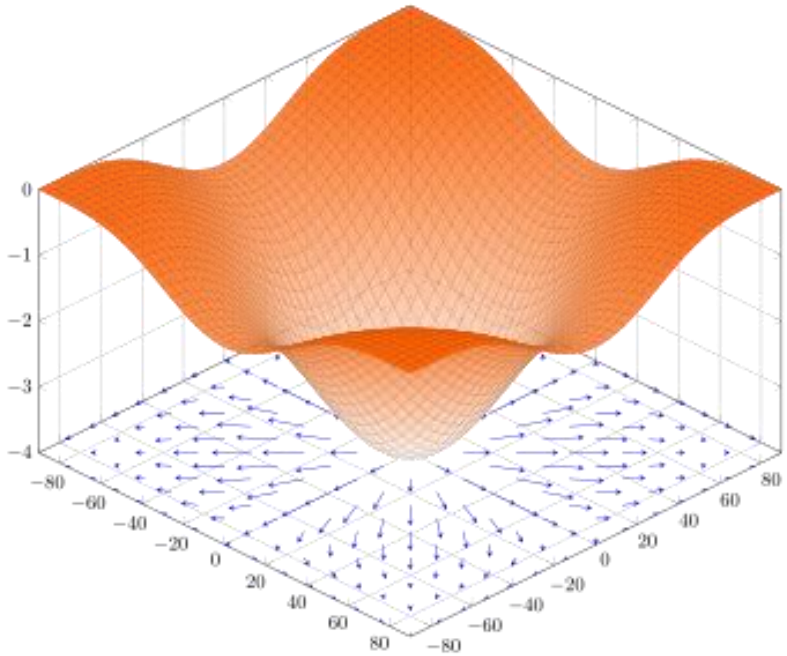
Newton's method



Gradient Descent

1-4. Optimization

What is Gradient?



$$\nabla f(p) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(p) \\ \vdots \\ \frac{\partial f}{\partial x_n}(p) \end{bmatrix}.$$

In vector calculus, the **gradient** of a scalar-valued differentiable function f of several variables is the vector field (or vector-valued function) ∇f whose value at a point p gives the direction and the rate of fastest increase

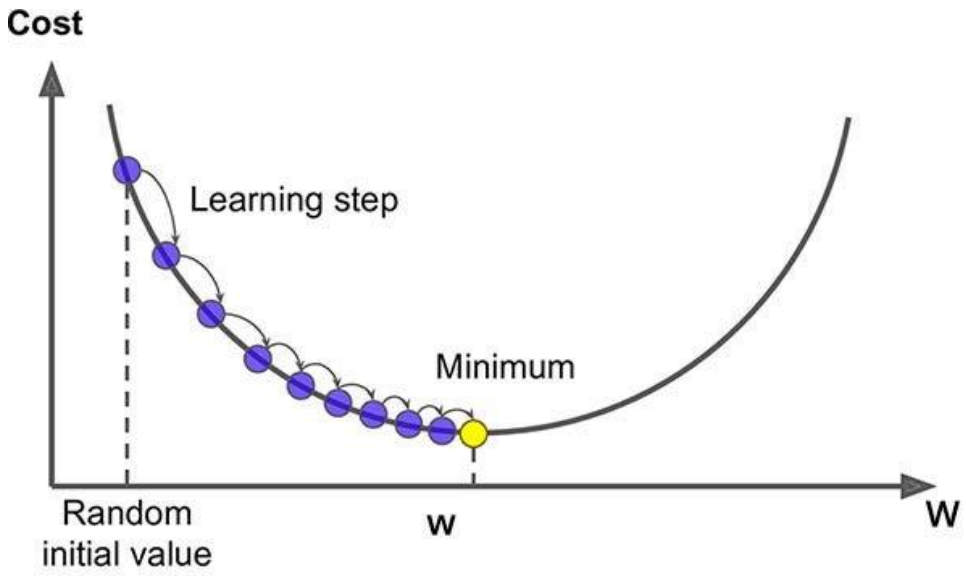
1-4. Optimization

Gradient Descent

Method: at each step, move in direction of negative gradient

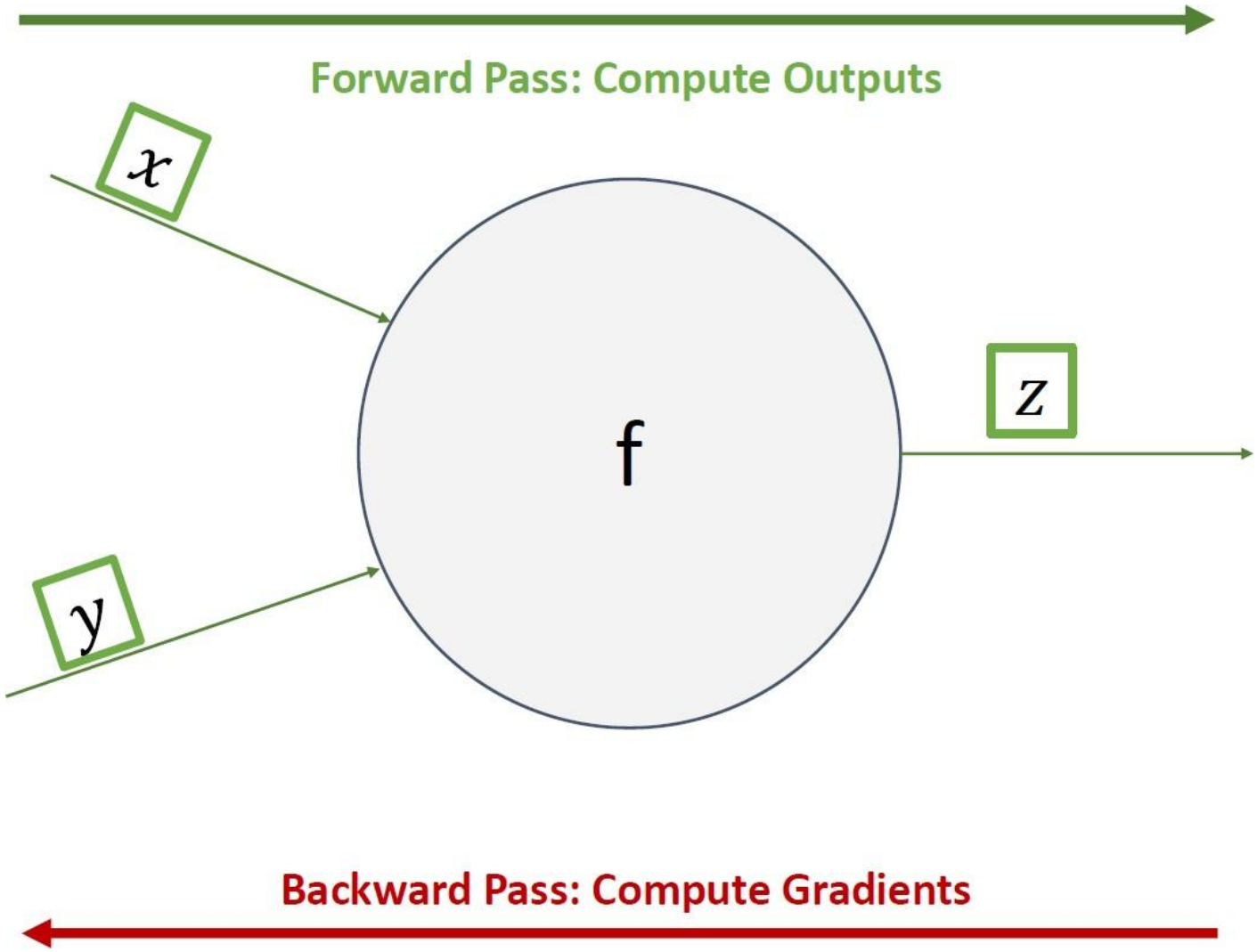
```
w0 = initialize()                    #initialize
for iter in range(numIters):
    g = ∇_w L(w)                     # eval gradient
    w = w + -stepsize*g              # update w
return w
```

Learning rate



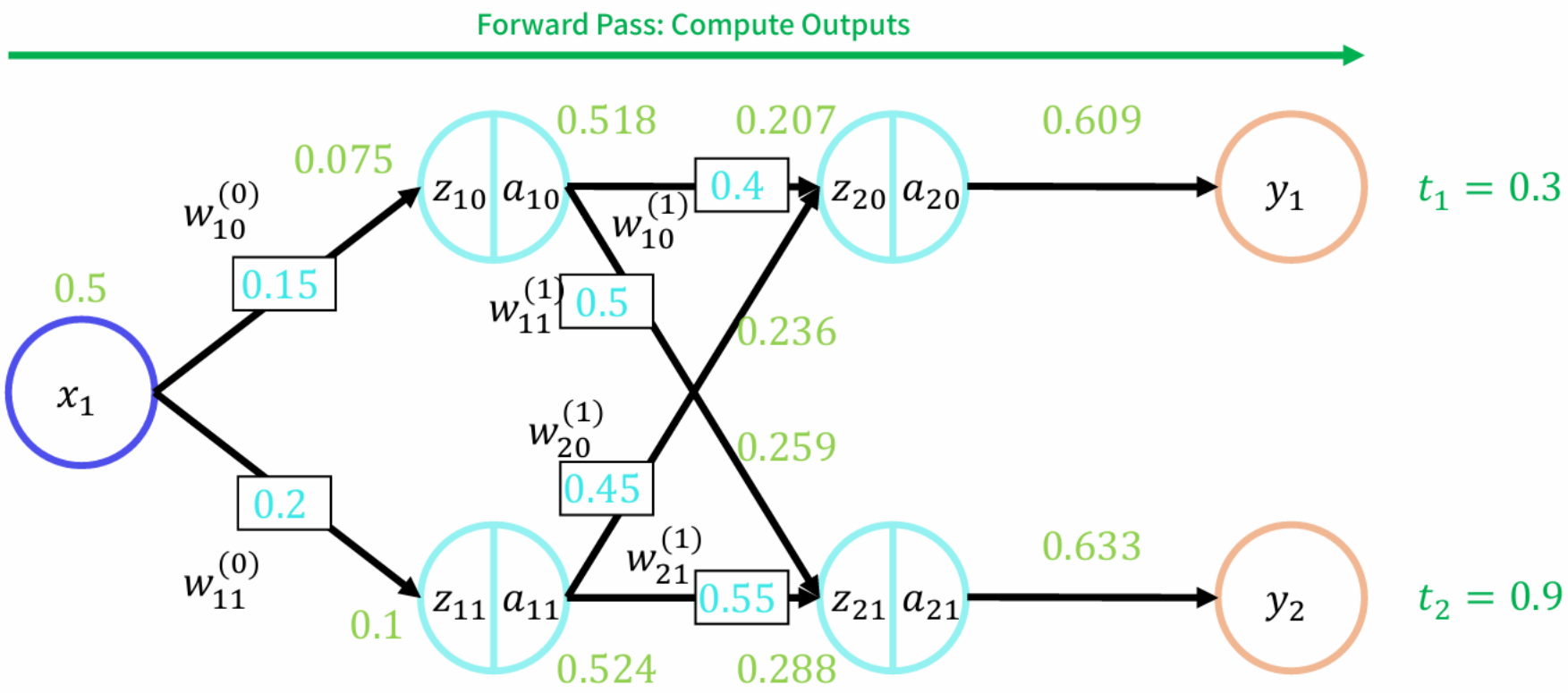
1-4. Optimization

Backpropagation



1-4. Optimization

Backpropagation

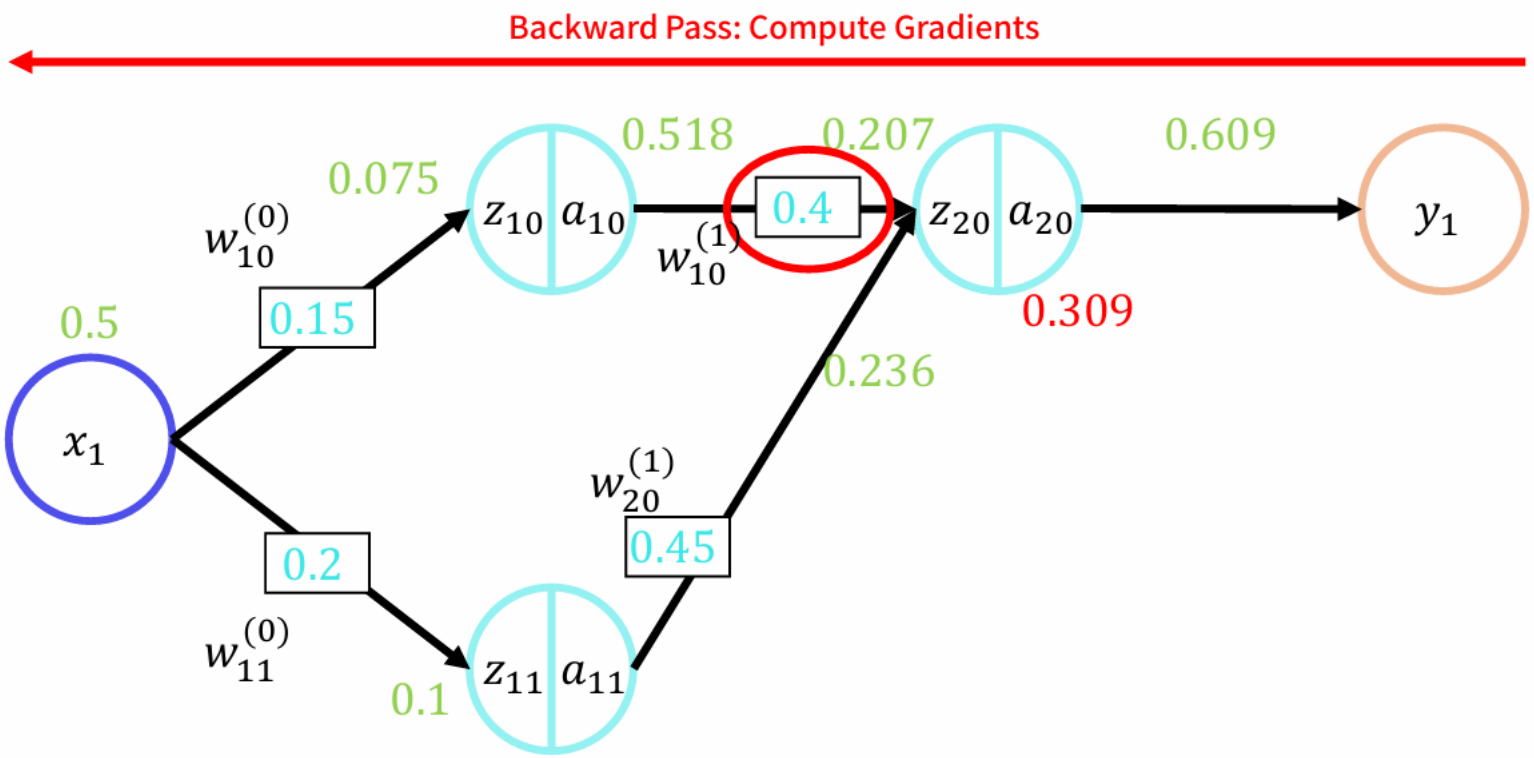


$$E = \frac{1}{2}\sum_{i=1}^{2}(t_i - y_i)^2$$

$$w_{i+1} = w - \eta * \frac{\partial E}{\partial w_i}$$

1-4. Optimization

Backpropagation



$t_1 = 0.3$

$$E = \frac{1}{2}\sum_{i=1}^{2}(t_i - y_i)^2$$

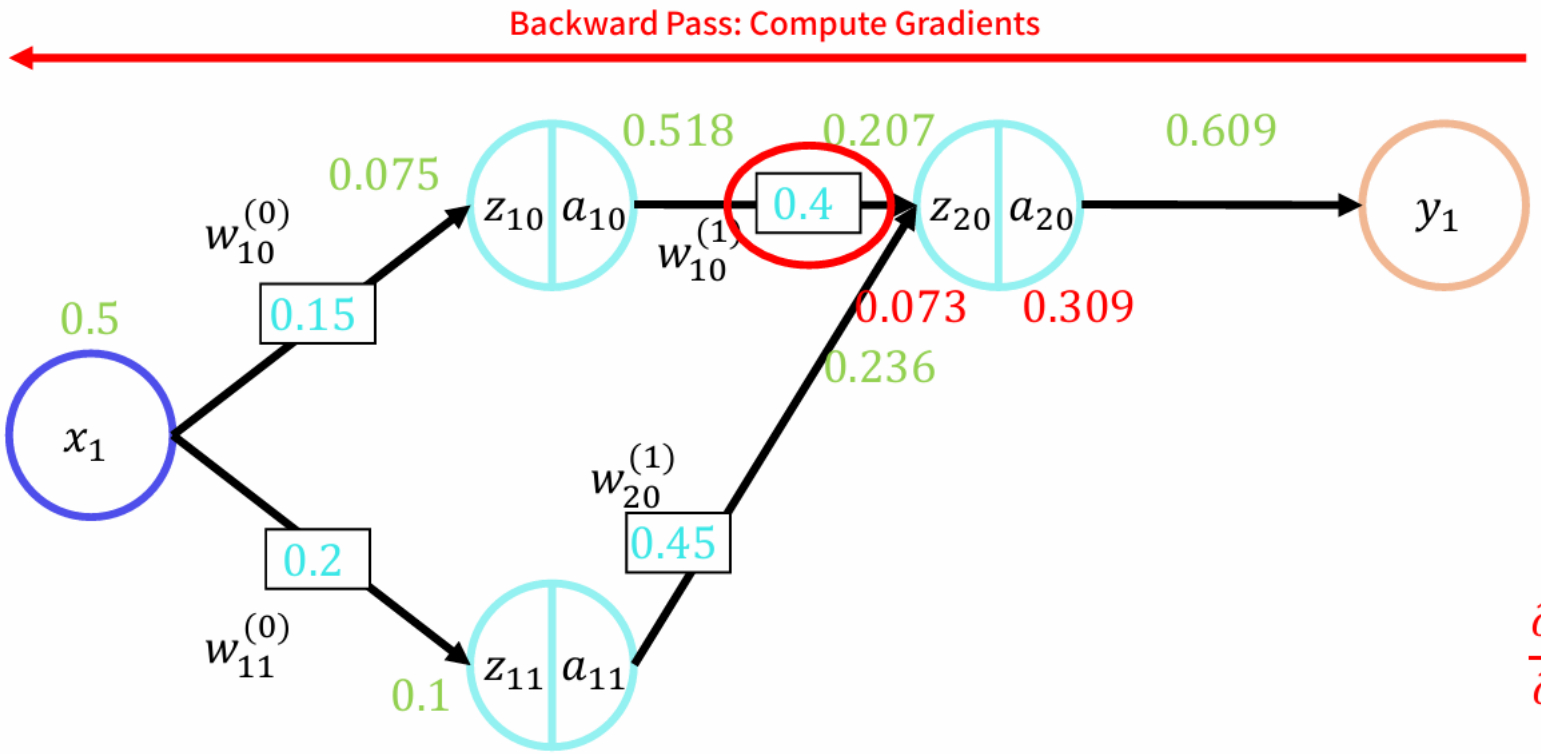
$$\frac{\partial E}{\partial w_{10}^{(1)}} = \frac{\partial E}{\partial a_{20}}\frac{\partial a_{20}}{\partial z_{20}}\frac{\partial z_{20}}{\partial w_{10}^{(1)}}$$

$$E = \frac{1}{2}\left((t_1 - a_{20})^2 + (t_2 - a_{21})^2\right)$$

$$\frac{\partial E}{\partial a_{20}} = -(t_1 - a_{20}) = 0.309$$

1-4. Optimization

Backpropagation



$t_1 = 0.3$

$$E = \frac{1}{2}\sum_{i=1}^{2}(t_i - y_i)^2$$

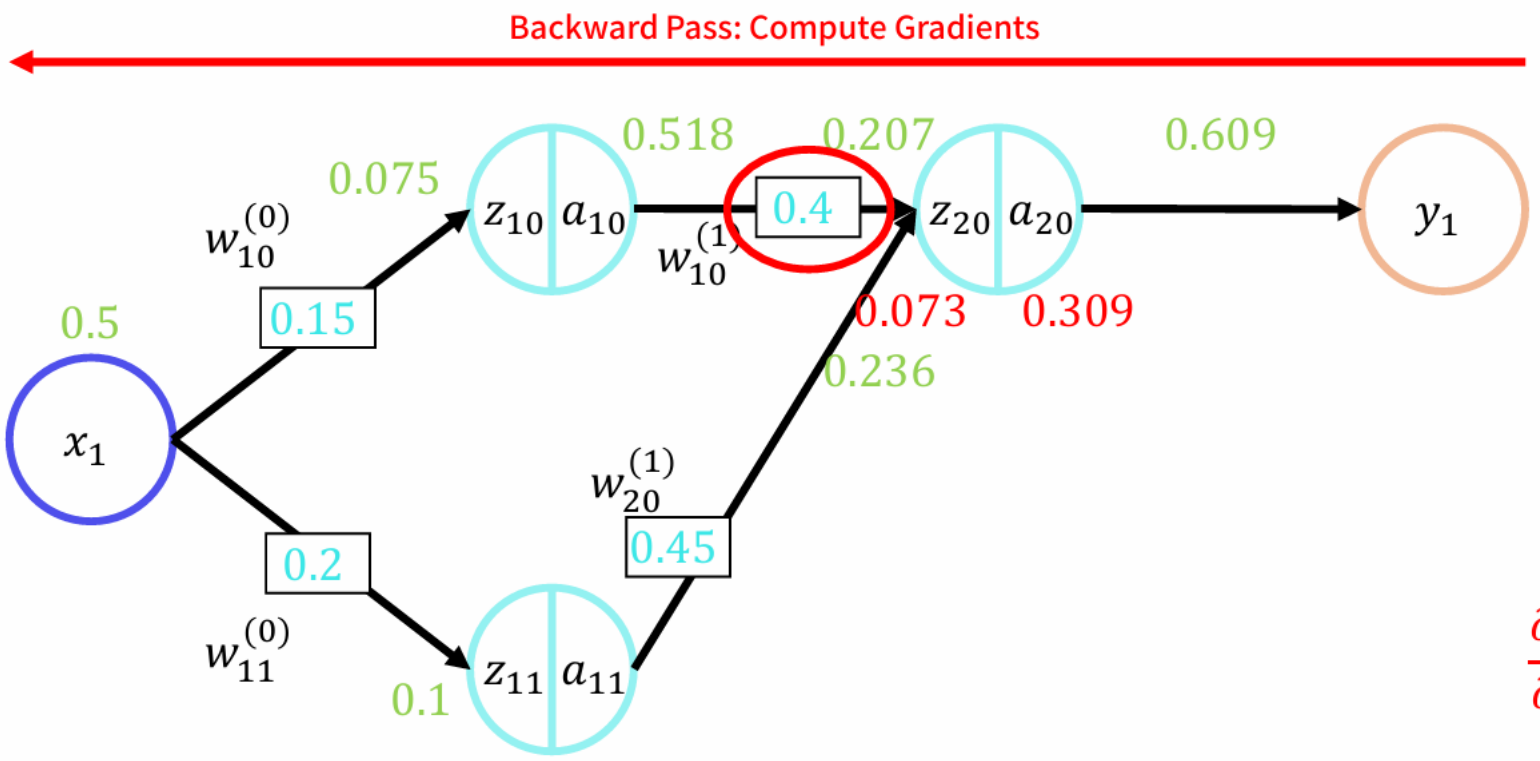
$$\frac{\partial E}{\partial w_{10}^{(1)}} = \frac{\partial E}{\partial a_{20}}\frac{\partial a_{20}}{\partial z_{20}}\frac{\partial z_{20}}{\partial w_{10}^{(1)}}$$

$$a_{20} = sigmoid(z_{20})$$

$$\frac{\partial a_{20}}{\partial z_{20}} = sigmoid(z_{20})\big(1 - sigmoid(z_{20})\big)$$
$$= 0.238$$

1-4. Optimization

Backpropagation



$t_1 = 0.3$

$$E = \frac{1}{2}\sum_{i=1}^{2}(t_i - y_i)^2$$

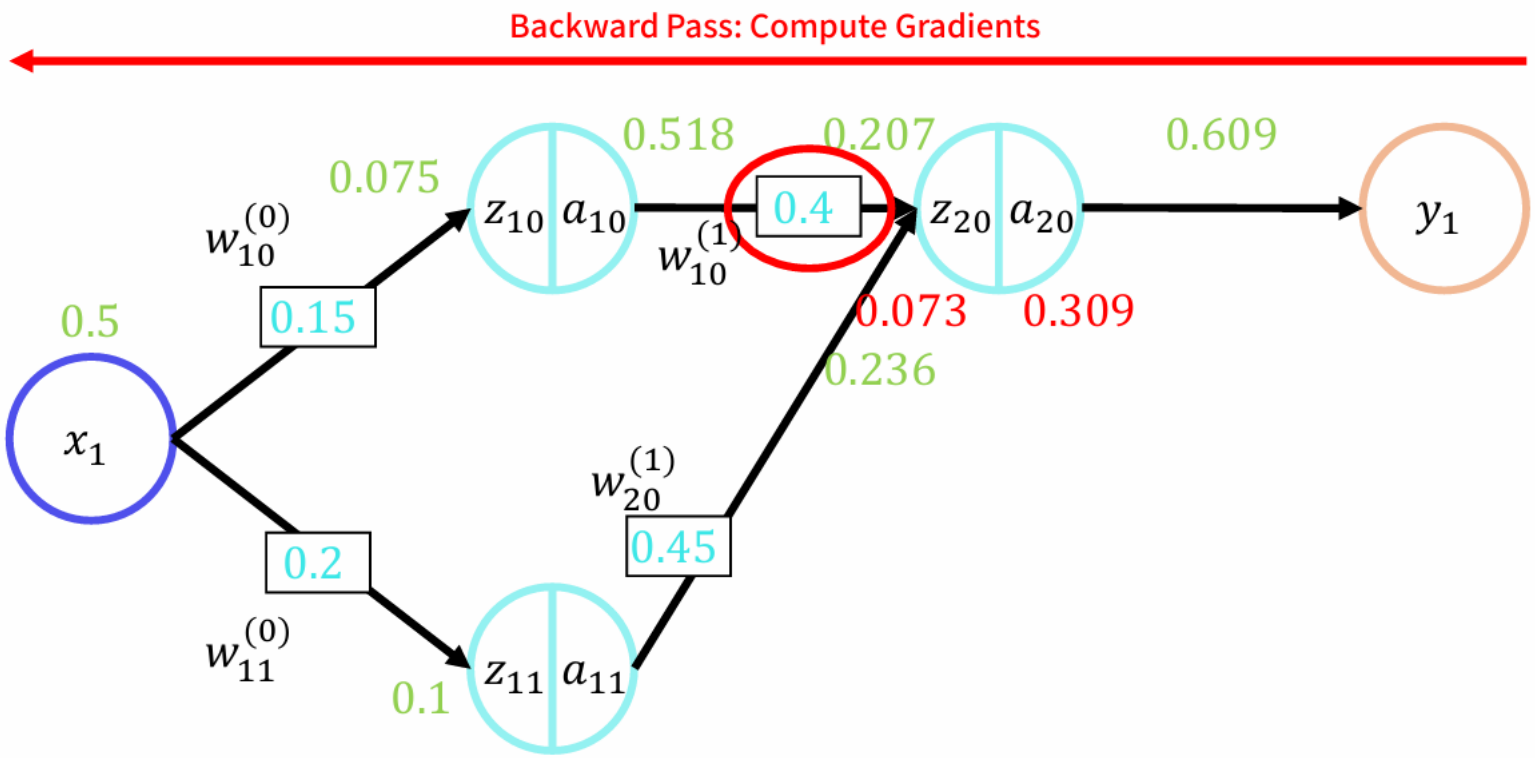
$$\frac{\partial E}{\partial w_{10}^{(1)}} = \frac{\partial E}{\partial a_{20}}\frac{\partial a_{20}}{\partial z_{20}}\frac{\partial z_{20}}{\partial w_{10}^{(1)}}$$

$$a_{20} = sigmoid(z_{20})$$

$$\frac{\partial a_{20}}{\partial z_{20}} = sigmoid(z_{20})\big(1 - sigmoid(z_{20})\big)$$
$$= 0.238$$

1-4. Optimization

Backpropagation



$t_1 = 0.3$

$$E = \frac{1}{2}\sum_{i=1}^{2}(t_i - y_i)^2$$

$$\frac{\partial E}{\partial w_{10}^{(1)}} = \frac{\partial E}{\partial a_{20}}\frac{\partial a_{20}}{\partial z_{20}}\frac{\partial z_{20}}{\partial w_{10}^{(1)}}$$

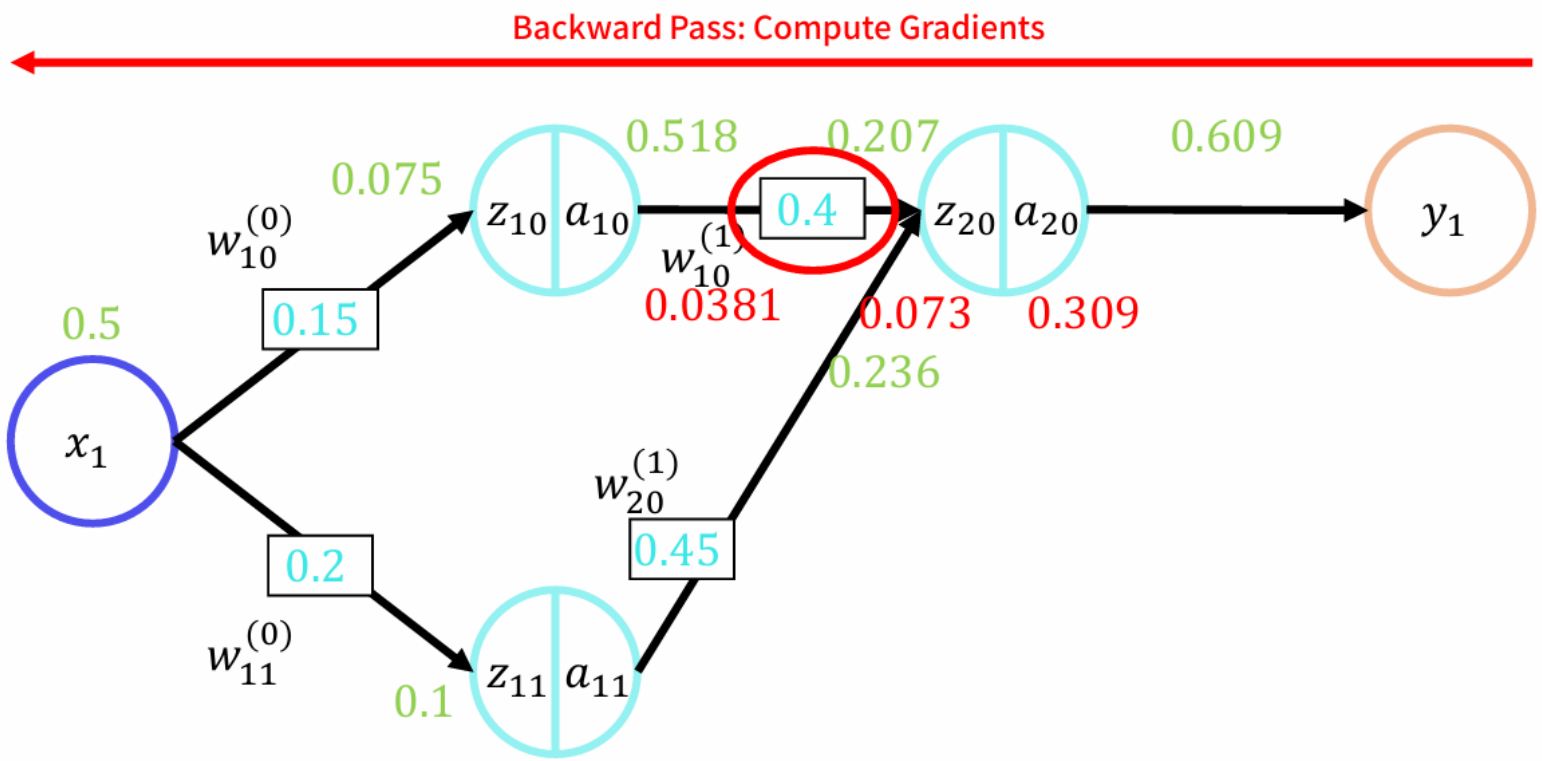
$$z_{20} = a_{10}w_{10} + a_{11}w_{11}$$

$$\frac{\partial z_{20}}{\partial w_{10}^{(1)}} = a_{10}$$

$$= 0.518$$

1-4. Optimization

Backpropagation



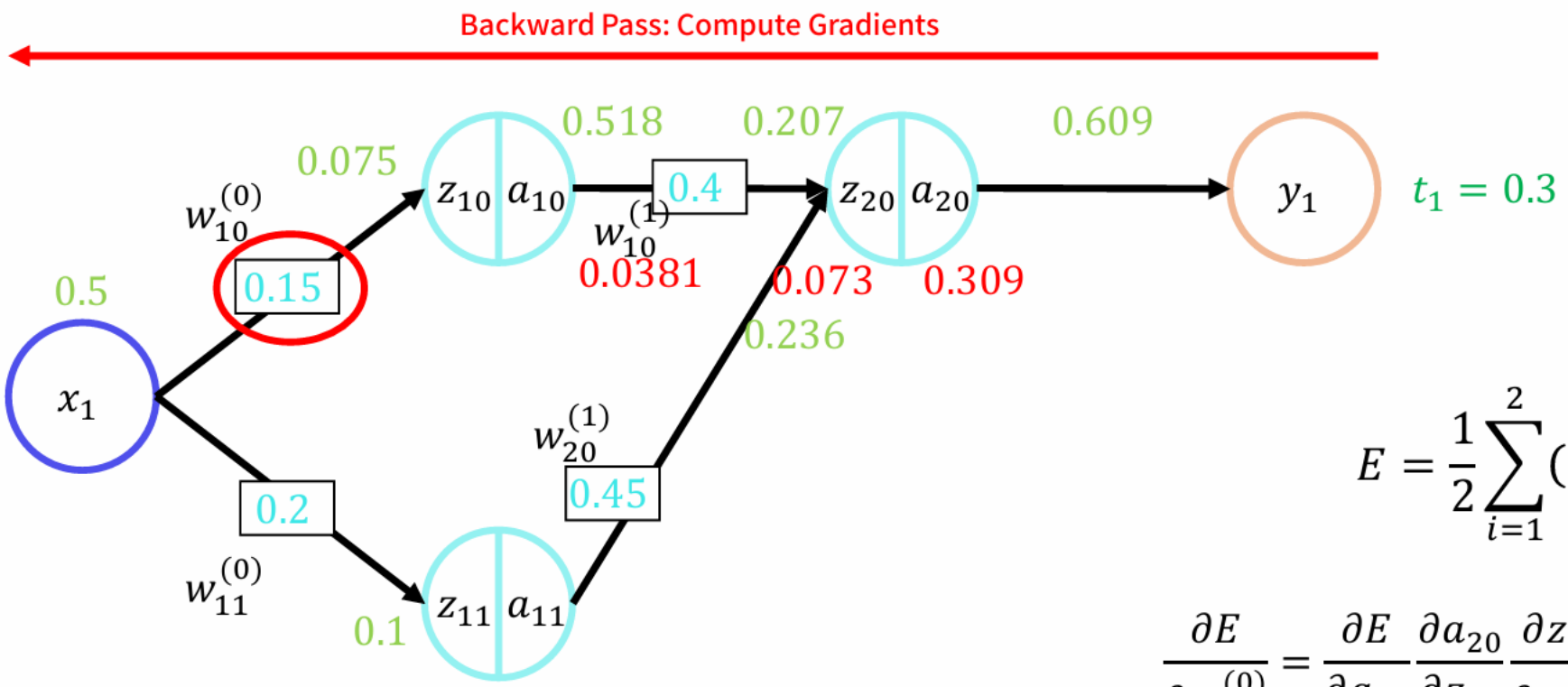
$$E = \frac{1}{2}\sum_{i=1}^{2}(t_i - y_i)^2$$

$$\frac{\partial E}{\partial w_{10}^{(1)}} = \frac{\partial E}{\partial a_{20}}\frac{\partial a_{20}}{\partial z_{20}}\frac{\partial z_{20}}{\partial w_{10}^{(1)}}$$

$$= 0.0381$$

1-4. Optimization

Backpropagation

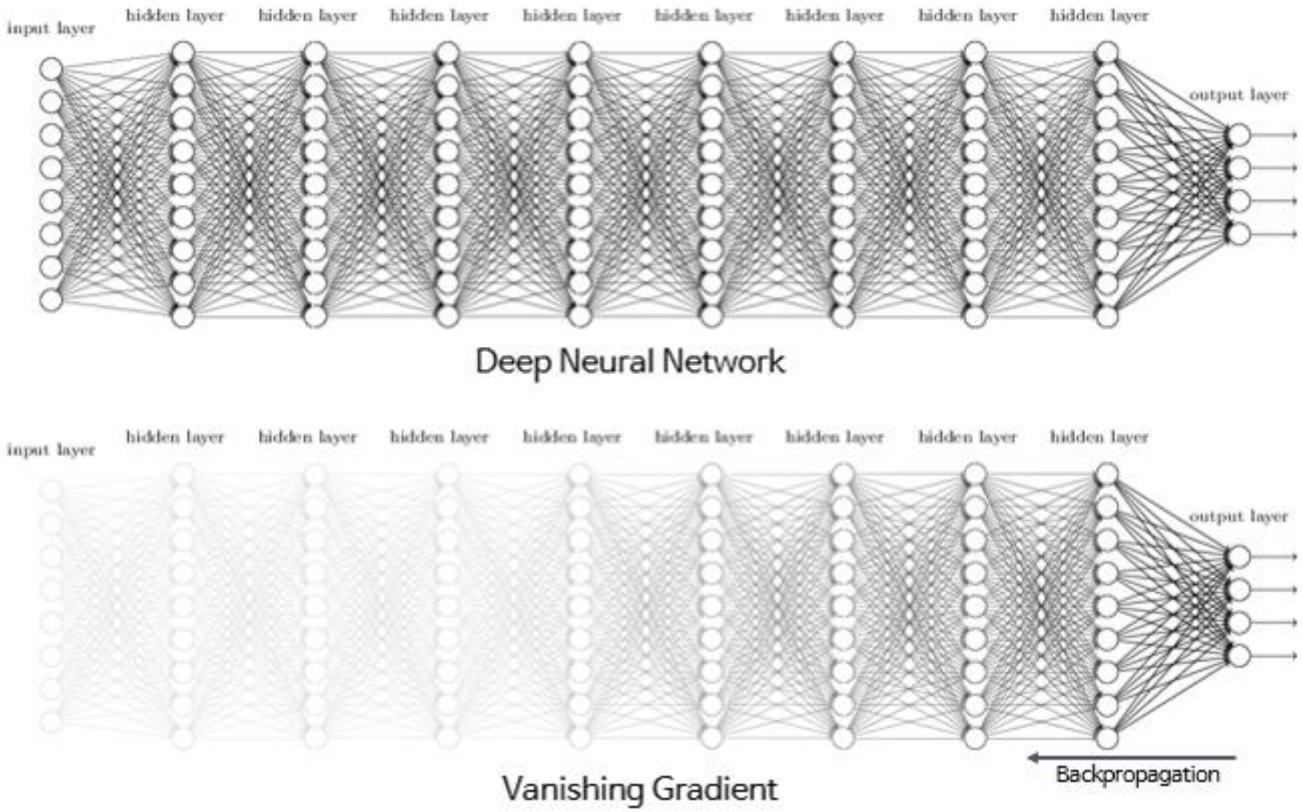


$$E = \frac{1}{2}\sum_{i=1}^{2}(t_i - y_i)^2$$

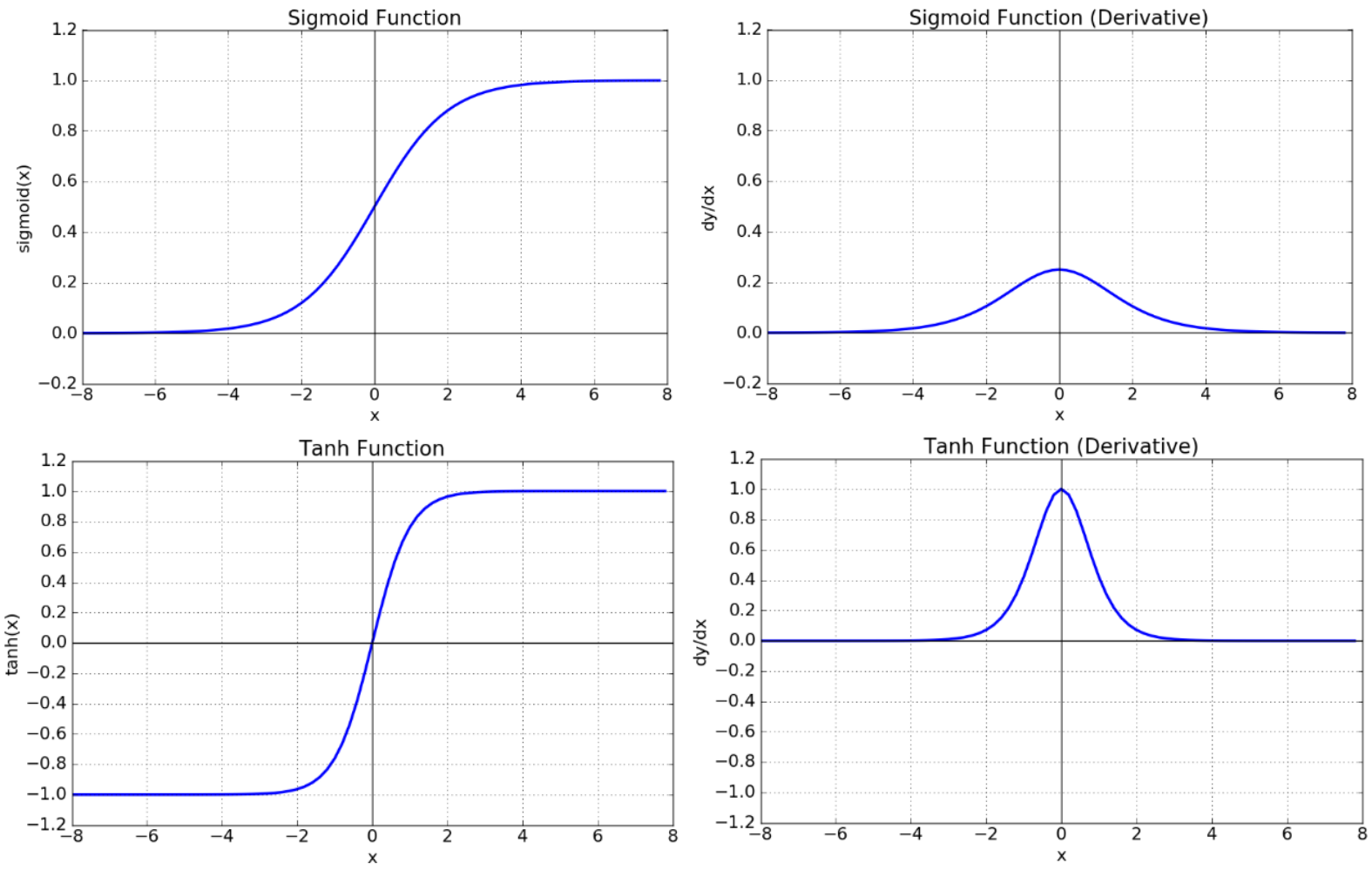
$$\frac{\partial E}{\partial w_{10}^{(0)}} = \frac{\partial E}{\partial a_{20}}\frac{\partial a_{20}}{\partial z_{20}}\frac{\partial z_{20}}{\partial w_{10}^{(1)}}\frac{\partial w_{10}^{(1)}}{\partial a_{10}}\frac{\partial a_{10}}{\partial z_{10}}\frac{\partial z_{10}}{\partial w_{10}^{(0)}}$$

1-5. Gradient Vanishing

as the network depth or sequence length increases, **<u>the gradient decrease</u>**, slowing the training process



1-5. Gradient Vanishing

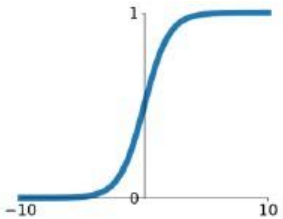


1-5. Gradient Vanishing

Activation Function (Non-Linearity)

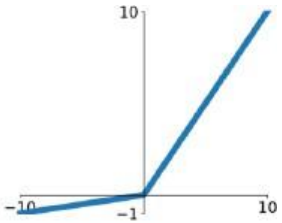
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



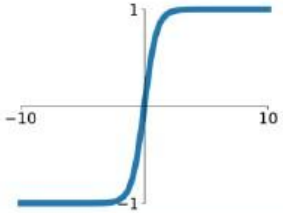
Leaky ReLU

$$\max(0.1x, x)$$



tanh

$$\tanh(x)$$

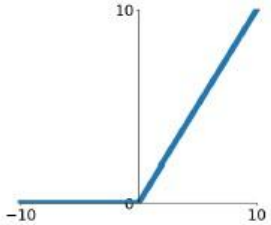


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ReLU

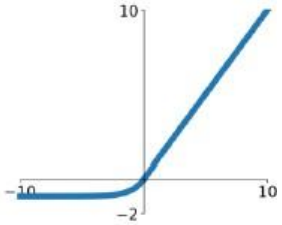
$$\max(0, x)$$



Dying ReLU?

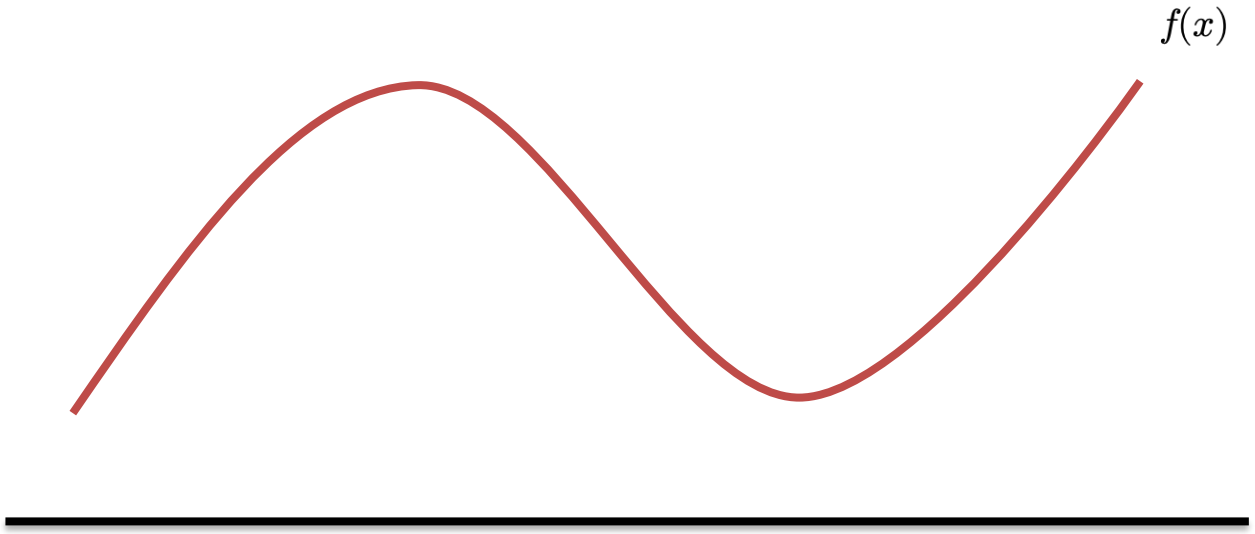
ELU

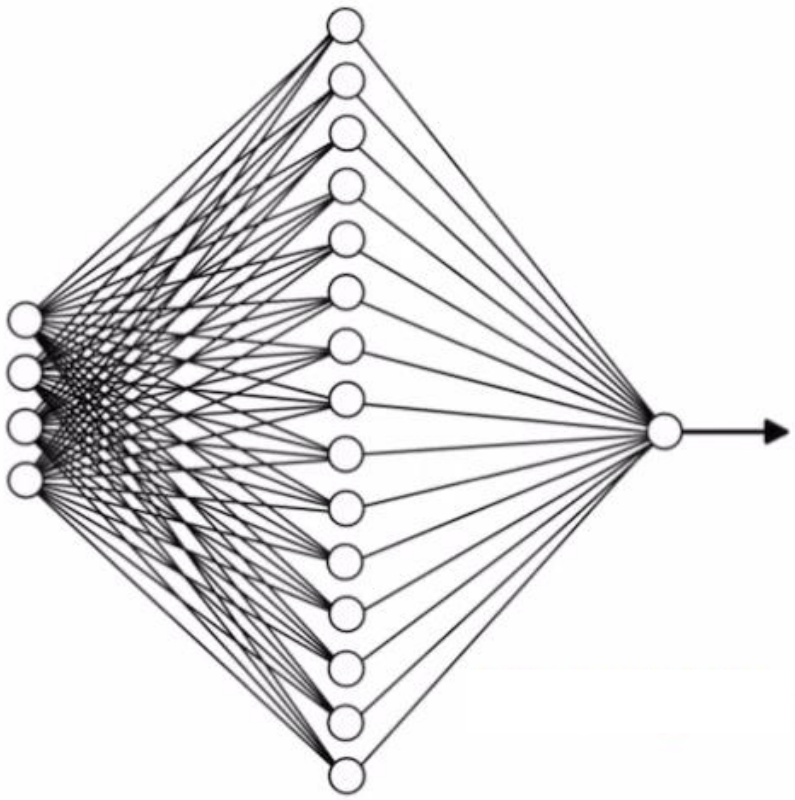
$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



1-6. Universal Approximation Theorem

Upper Bound of Neural Network

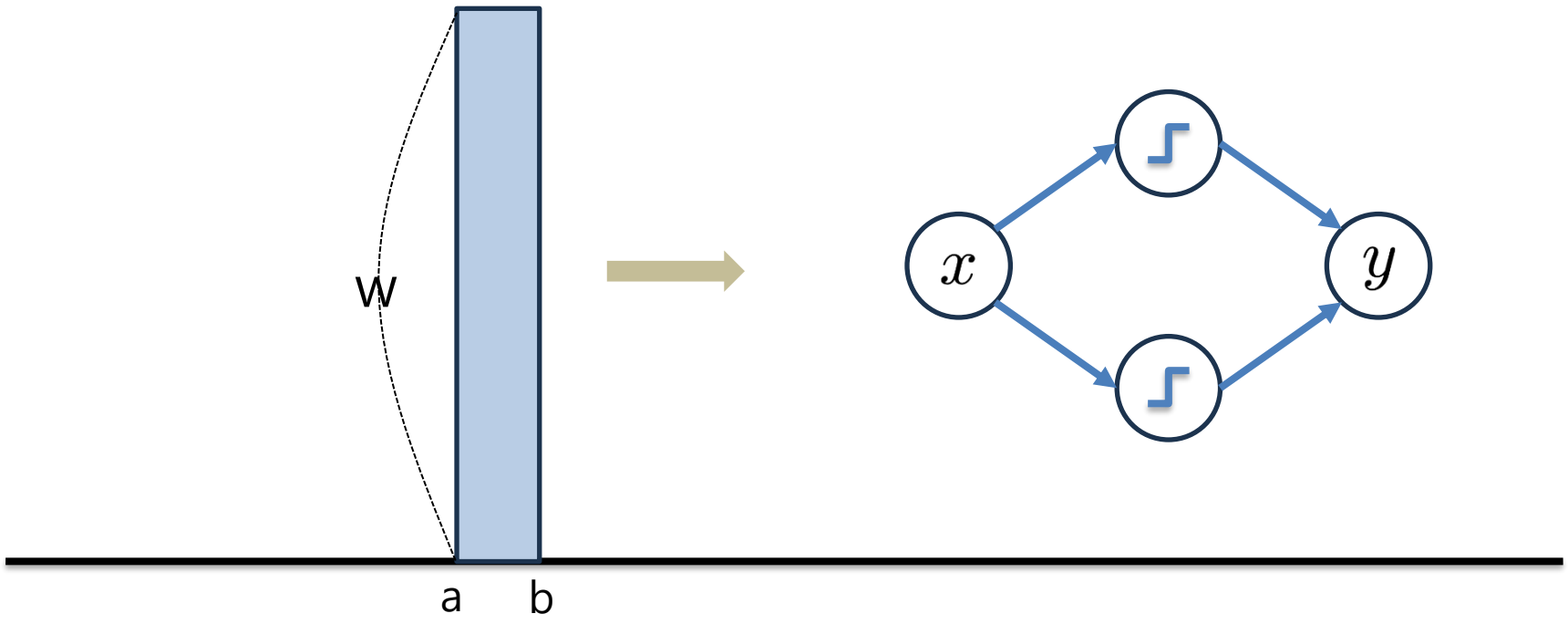




1-6. Universal Approximation Theorem

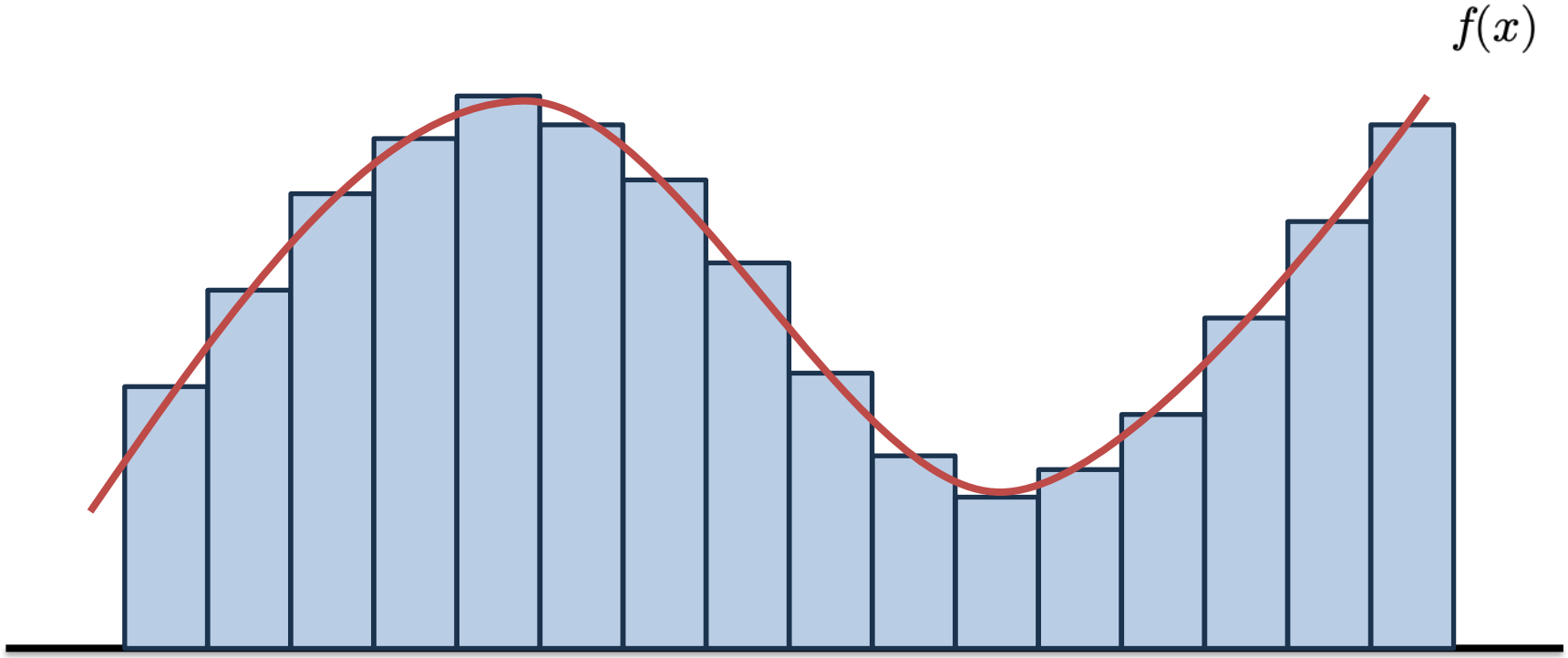
Universal Approximation Theorem

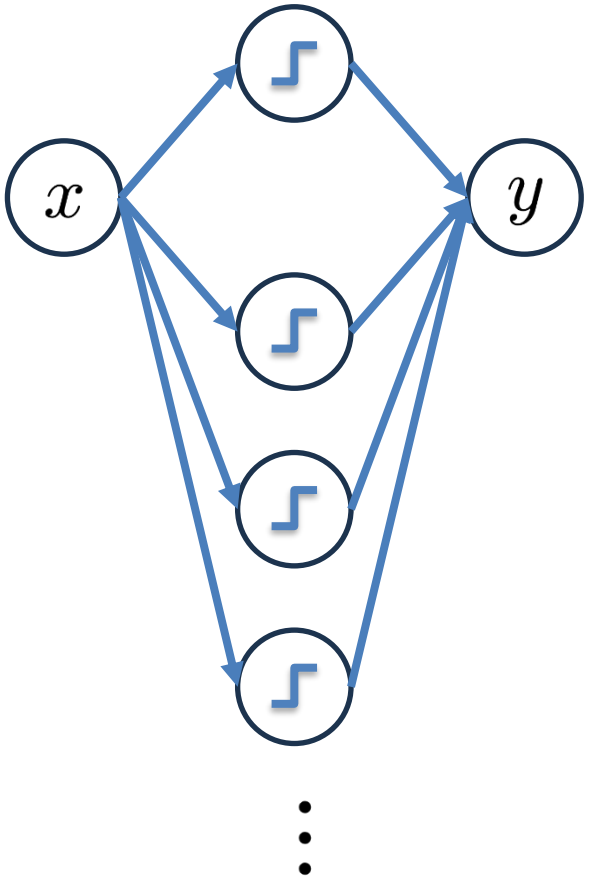
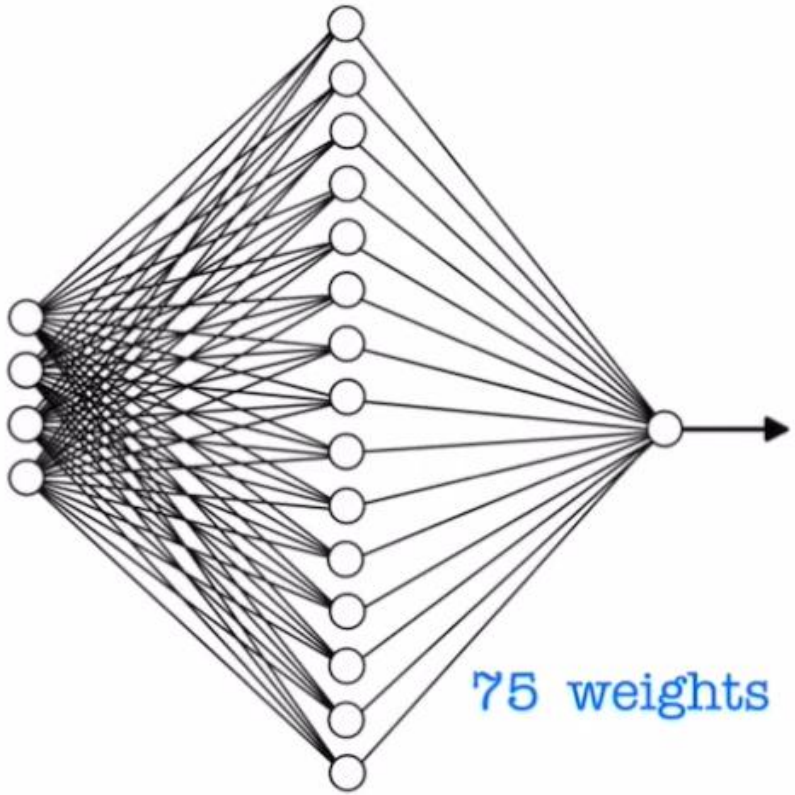
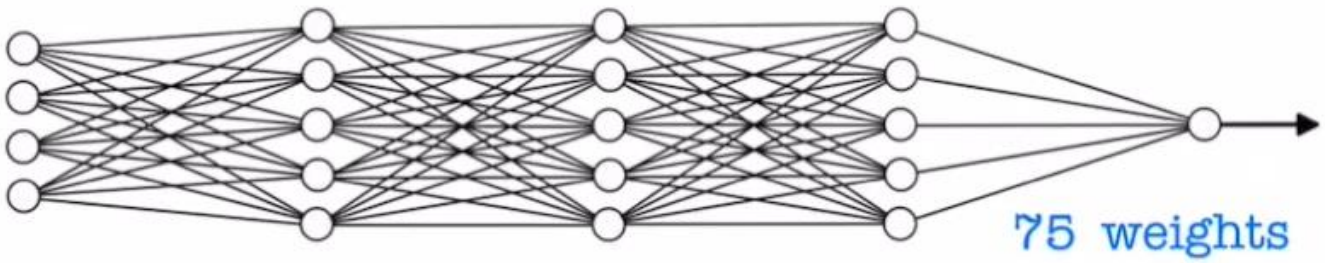
$$w \times (\delta(x-a) - \delta(x-b))$$



1-6. Universal Approximation Theorem

Universal Approximation Theorem





1-6. Universal Approximation Theorem